数据中心的“税”

从 RPC 微服务到 Agent：负载洞察与鲲鹏的软硬件发力点

主问题：鲲鹏 950（软件）和 960（硬件）该在哪些通用开销上发力，各由哪个团队做？

基于约 75 篇论文（2015–2026）的综述，完整版 v3（团队学习 / 评审用）：全篇按“一张矩阵 + 一个乘法”组织
配套：REPORT.md · INSIGHT_DATACENTER_TAX_AI_ERA.md · ~/AgenticAI 阅读报告 · 逻辑提案 OUTLINE_RPC_DECK_V3_LOGIC.md；图均截自论文原文

Kunpeng 基础加速 / 洞察工作组 · v3 · 2026-10-16

一页结论：税是数据跨边界的成本——按操作看是数据中心税，按边界看 RPC 税是其中一列； Agent 给这张表加了三列

现象	数据中心没有杀手应用； CPU 的通用开销是一批共享操作（数据中心税）。 Google 2015 年六项合计 22–27%，另有内核约 18–20%（fleet 平均）[01]。	第 8–11 页
成因	这些操作是数据跨边界的代价。业务拆成数百个服务、存储与计算分开： Meta 一次 Fetch 请求中位经过 498 个服务块 [03]。部署形态再把每次跨越放大： sidecar 让每次 RPC 的 CPU 变成 2.7–3.7 倍 [21][18]。	第 13–20 页
对账	按操作切得到数据中心税，按边界切得到 RPC 税等，两者只在交叉格重叠：序列化 9.6% 里只有约 1.2 个百分点在 RPC 上 [25][02]。两本账有五处对得上；压缩那一格对不上，所以 RPC 税要说口径： [02] 口径 7.1%，严格按 RPC 路径约 4–5%（推算）。	第 21–24 页
削减	四个杠杆：减次数、降系数、降每次固定开销、降每字节开销。放的位置决定能吃多少：贴核吃整行（序列化 9.6%、压缩 2.9% 全覆盖），网卡只吃 RPC 那一格（序列化约 1.2%）。	第 27–36 页
Agent	同一张表多了三列：模型、执行环境、会话状态。每次调用的单价随上下文变长而上涨，还多了“内存 × 时间”这种货币；先撞上的是内存容量、单核速度和沙箱启动，不是峰值算力。	第 38–52 页
鲲鹏	行 → 核内指令与 KAE；列 → UB 与 DPA；次数和系数 → 950 软件与协作团队；内存 × 时间 → 内存系统。本团队 P0： 增量 tokenizer、 KAE 的 4 KB 压缩（快照与 zswap）、 DPA 接口； 960 的第一诉求是前端取指。	第 54–63 页

读法：第一部分按行看现象，第二部分按列找成因并对账，第三、四部分讲四个杠杆，第五部分给表加列，第六部分按行和列落到硅上。

数据来源与口径：各条的数字与口径见右侧页码；“行 / 列 / 杠杆”是本报告的组织方式（第 6 页），不是某篇论文的原话；“约 4–5%”的推算见第 21 页。

阅读地图：每一部分回答一个问题，再把下一个问题交出去

六个部分的问题链（→ 是交给下一部分的问题）

一 现象（按行看）： CPU 的周期花在哪？没有杀手应用 → 税的组成 → 存储平台 → 微架构。→ 这些操作为什么被调这么多次？

二 成因（按列看）： 业务怎么拆、一次请求经历了什么？业务形态 → 请求的一生 → 跨越次数 → 每次跨越的形状 → 形态系数 → 对账。→ 每个因子能削多少？

三 削减： 四个杠杆（减次数、降系数、降每次固定开销、降每字节开销）；学界各削哪一项、吃行还是吃列。

四 综合： 每一格多大、已证明能削多少；文献的需求清单；客户类型差异。→ AI 来了，这张表还成立吗？

五 AI 与 Agent： 为什么现在要看（市场信号）→ 一次 Agent 请求的一生 → 表上多了三列和一种货币 → 三个专题对应三条新列。→ 鲲鹏在每一格上做什么？

六 规划： 行 → 核内指令与 KAE；列 → UB 与 DPA；次数与系数 → 软件与协作团队；内存 × 时间 → 内存系统；矩阵的空格 → 950 待测清单。

论文覆盖（约 75 篇，其中 2024–2026 年的约 35 篇）

论文类别	篇数	年份
负载刻画 / benchmark （Google 、 Meta 、阿里、 Uber ）	16	2015–2025
微服务部署形态（ mesh 、 RPC 服务化、 FaaS ）	10	2019–2026
RPC 通信栈（传输、内核旁路、调度、硬件卸载、 CXL ）	20	2018–2025
RPC 序列化与压缩（软件、零拷贝、加速器）	11	2020–2026
面向微服务的 CPU / SKU 设计	3	2019–2024
AI 与 Agent 负载（第五部分的核心论文）	14	2025–2026

数据来源分三层： 生产测量（ Google 、 Meta 、阿里、 GitHub Copilot 、 DeepSeek ）→ 学术 benchmark → 原型评估。三层数字可差一个数量级，逐页标口径。

一致性约定： 每个对象只有一个主讲页，其它页引用同一条表述；附录 B 是关键数字的口径表。 [NN] 是 notes/ 笔记编号， [L1–L4] 、 [R1–R10] 是 ~/AgenticAI 报告的论文编号。

“业务特征 → 负载特征 → 软件（ 950 ）→ 硬件（ 960 ）”这条链全篇走两遍：第一至四部分走传统负载，第五部分走 Agent ，第六部分合流。

先把词说清楚

术语速查：后面每一页都会用到的十二个词

边界与“跨边界的税”

数据从一处到另一处（另一个服务、磁盘、进程，AI 时代还有模型、沙箱）要做格式转换、压缩、拷贝、加密、走协议栈、唤醒线程。本报告把这些开销看成跨边界的税。

行与列（税的矩阵）

行 = 操作（序列化、压缩、拷贝……），按行加是数据中心税；列 = 边界（服务、存储……），按列加是某条边界的税。 RPC 税是“服务边界”那一列。

Datacenter tax （数据中心税）

跨应用共享、与业务逻辑无关的底层例程耗掉的 CPU 周期：序列化、压缩、内存分配、拷贝、哈希、网络栈与 RPC 库等。Google 2015 年命名 [01]。

RPC 与 RPC 税

服务 A 像调本地函数一样调服务 B。 RPC 税 = 为跨服务边界花的周期：通信专属操作（网络栈、 RPC 库） + 序列化、压缩、加密在 RPC 上的那几格。

fleet （机群） / fleet 平均

一家公司全部服务器的总体。“ fleet 平均 x%”指全部 CPU 周期里的 x%；“某热点服务里 y%”是另一个口径，两者常差一个数量级。

序列化 / 反序列化

把内存对象变成字节流 / 再变回来（ protobuf 、 Thrift 、 JSON ）。不只发生在 RPC 上：存储读写、日志、进程内传递也用，而且用得更多。

部署形态与形态系数

RPC 栈编进应用进程（ library ）还是旁挂代理进程（ sidecar ）。形态系数 = 一次逻辑跨越实际变成几次物理跨越： sidecar 每 RPC 多穿两次内核协议栈。

四种货币

每次跨越的代价用四种单位计： CPU 周期（决定机器数）、延迟（决定 SLO ）、字节（决定带宽）、内存 × 时间（决定沙箱密度， AI 时代新增）。

P50 / P99 / 尾延迟

一批请求的延迟从小到大排序，中位数 / 第 99 百分位；“尾延迟”指最慢的那一小部分请求，通常由排队和干扰决定。

IPC / MPKI / 前端 stall

每周期完成的指令数 / 每千条指令的 cache 未命中次数 / 流水线因取不到指令而空转的比例（ Top-Down 方法的 front-end bound ）。

内核旁路 / 零拷贝

应用绕过操作系统内核直接操作网卡（ DPDK 、 RDMA ） / 数据从应用到网卡不在内存里多搬一遍。

卸载（ offload ）

把某段工作从 CPU 核搬到网卡、 DPU 、片上加速器上做；关键变量是“放在哪”（决定吃到一行还是一列）和“调用一次的固定开销”。

数据来源与口径：术语定义综合 [01][02][23] 与本报告第 0 章；黄底四个词是 v3 的组织框架（第 6 页），是本报告的说法，不是论文原话。

AI 与 Agent 术语：第五、六部分会用到的十个词

token 与 tokenize （BPE）

模型处理文本的最小单位，约 4 字节一个。 tokenize 把文本切成 token 编号；主流算法 BPE 先用正则预分词，再按合并表反复合并相邻符号。

prefill 与 decode

模型先把整段输入读一遍（prefill，算力密集），再一个 token 一个 token 地生成输出（decode，访存密集）。

KV 缓存与前缀缓存命中

模型读过的上下文留下的中间状态叫 KV 缓存。下一次调用时，前面没变的部分直接复用，叫前缀缓存命中。

会话、请求、步

用户打开到关掉是一个会话；每输入一句指令是一次请求；一次请求循环多轮，每轮“调一次模型 + 执行工具”是一步（TraceLab 定义）。

harness （Agent 主循环）

调用模型、解析工具调用、拼装上下文的程序，如 Claude Code 、 Codex 的 CLI；通常跑在用户电脑或云端 VM 上。

工具调用与 MCP

模型要求运行命令、读写文件、搜索， harness 照做并把结果塞回上下文。 MCP 是调用外部工具服务器的协议，消息是 JSON-RPC 。

沙箱的三档

进程级限制（bubblewrap 、 seccomp 、 Seatbelt ）；容器（Docker， gVisor 是在用户态重做内核的特殊容器）；微虚拟机（microVM，如 Firecracker ）。

冷启动

从零建出进程、解释器、容器或 microVM 的固定开销：建页表、清零内存、动态链接、解释器初始化，以及冷的分支预测器和 TLB 。

状态驻留（活着但空闲）

会话没结束、但在等人或等工具时， KV 缓存、沙箱内存和会话状态一直占着内存。它的单位是“ GB × 时间”，不是 CPU 周期。

RL rollout

强化学习后训练时批量跑成千上万个 Agent 任务来产生训练数据；每条轨迹一个沙箱，用完即弃。

数据来源与口径：综合 INSIGHT_DATACENTER_TAX_AI_ERA.md 0.4 节术语表、 [56] TraceLab 的会话 / 请求 / 步定义、 ~/AgenticAI 《 Agentic 时代的数据中心》的沙箱分类。

全篇的骨架：一张矩阵加一个乘法——按行加是数据中心税，按列加是某条边界的税， RPC 税是服务那一列

操作 ↓ ／边界 →	服务（RPC）	存储	进程 / 内核	模型	执行环境	会话状态
序列化 / 解析	●	●	○	●	○	
压缩	○	●	○		○	●
拷贝与搬移	●	○	○	○	○	●
内存分配	○	○	○	○	○	
哈希		●	○	●		
加密	●	○			○	
专属操作	通信：网络栈、RPC 库	文件格式库	系统调用、切换	tokenize 、推理调度	冷启动	驻留

● 主要发生处　○ 也会发生（示意）　黄列 = 服务边界　紫列 = AI 时代新增的三条边界

横着加一行 = 数据中心税的一项：如序列化整行 9.6%（Google fleet）[25]。→ 第一部分
竖着加一列 = 某条边界的税：服务那一列就是 RPC 税。→ 第二部分
交叉格 = 两种说法重叠的地方：如“序列化 × 服务”约 1.2% [02]。两本账能不能对上，见第 21–24 页。
第三种切法：按流水线状态（Top-Down）看每一格在核上怎么跑（第 11 页）。

税 ≈ Σ 边界 [跨越次数 × 每次单价 × 形态系数]

每条边界（矩阵的每一列）一项；单价用四种货币计

跨越次数

谁决定：业务怎么拆。
Meta 一次 Fetch 请求中位 498 个服务块 [03]；编码 Agent 每次请求调 8.8 次模型、 10.8 次工具 [56]。

每次单价

谁决定：做哪些操作 × 数据多大。
固定开销（库逻辑、唤醒、内核穿越）主导中位： RPC 请求中位 <1.5 KB [02]；每字节开销主导几百 KB 的尾部。

形态系数

谁决定：部署形态。
一次逻辑跨越变成几次物理跨越： sidecar 每 RPC 的 CPU 2.7–3.7× [21][18]；Agent 时代是隔离形态系数（进程 / 容器 / microVM）。

单价用四种货币付

- **CPU 周期** → 机器数
- **延迟** → SLO 与尾延迟
- **字节** → 网络与存储带宽
- **内存 × 时间** → 沙箱密度（AI 新增）

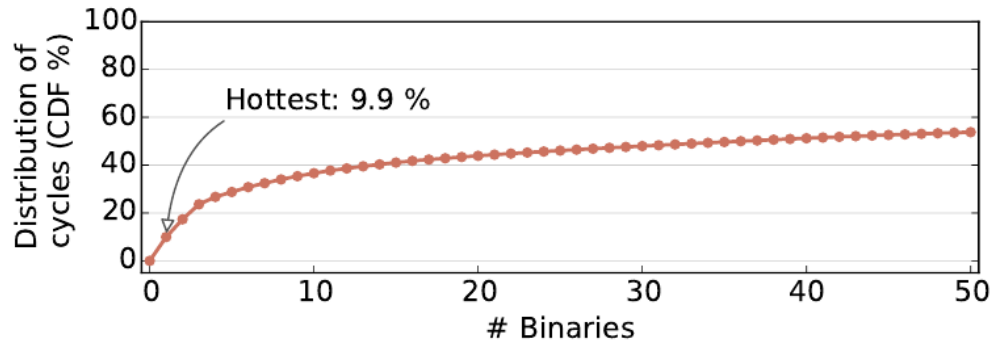
数据中心税是行， RPC 税是列，两者只在交叉格重叠。全篇只用这一张表和这一个乘法； Agent 没有换表，只是加了三列和一种货币。

数据来源与口径：矩阵与公式是本报告的组织方式（OUTLINE_RPC_DECK_V3_LOGIC.md 第 2 节），不是论文原文；●／○ 为示意，各格的实测数字见第 21 页（传统）和第 47 页（AI）。

现象： CPU 的周期花在哪（按操作看）

没有杀手应用 → 税的组成：矩阵的“行合计” → 存储 / 大数据平台：税更重 → 第三种切法：税在流水线上的样子
留给第二部分的问题：这些操作为什么被调这么多次？是为跨哪条边界调的？

按 CPU 周期看没有杀手应用：最热的单个程序只占约 10%，前 50 个合计约 60% 且逐年下降



Google 机群 CPU 周期按二进制累计分布：最热者 9.9%，前 50 个约 60%（[01] Fig. 1）

≈10%

最热的单个二进制占 fleet 周期

Google 2015，≈ 2 万台机器 [01]

≈60%

前 50 个二进制合计

且每年下降约 5 个百分点 [01]

6.3%

搜索叶子节点最热的函数

353 个函数才覆盖 80% 周期 [01]

≈50%

malloc 周期中 top-50 二进制覆盖率

Google 2024 [35]

按 CPU 周期，负载极度分散，而且越来越分散。从 Google 2015 的二进制分布到 2024 年的 malloc 归因，头部程序的覆盖率始终只有一半左右，而且每年下滑。应用内部也一样：搜索叶子节点最热的函数只占 6.3%。

注意度量：按调用次数看，RPC 集中在少数方法和服务上（第 16 页）。两者量的不是同一个东西，调用集中不等于周期集中。

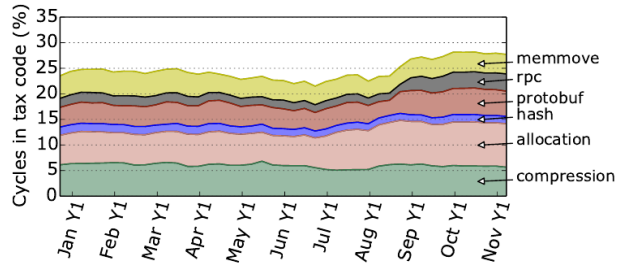
因此硬件专用化的目标是跨应用共享的底层例程，也就是“税”：序列化、压缩、内存分配、拷贝、哈希、网络栈与 RPC 库。它们在每个程序里都不大，加起来是机群里最大的可寻址块；下一页按操作（矩阵的行）列出它们。

结论：为单个应用优化的硬件没有 fleet 级价值；只有跨应用共享的“税”才值得硬件专用化——这是全篇的出发点。

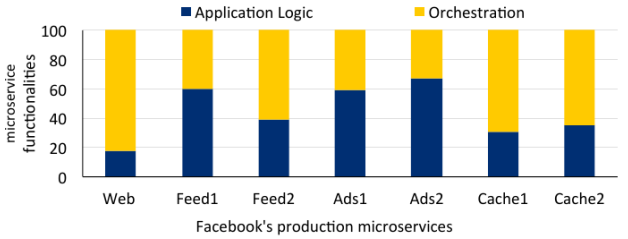
数据来源与口径：[01] Kanev 等，ISCA'15，Google ≈2 万台 Ivy Bridge 机器、3 年 GWP 采样，按二进制归因全部 CPU 周期（fleet 平均口径）；[35] Google ASPLOS'24，malloc 周期按二进制归因。

税的组成（按操作看，即矩阵的行）： Google 2015 年六项合计 22–27%； 2021–24 年的分项测量里序列化 9.6% 最大，分配降到 4.3%

操作（矩阵的行）	Google 2015 [01]	Google 2021–24 （全口径 = 行合计）	Meta 2020 [20] （7 个重要微服务）	备注
通用操作：每条边界都会用（序列化、压缩、拷贝、分配、哈希、加密）				
序列化（protobuf）	≈3–4%	9.6% （全部 protobuf 操作， C++ 占 88%；可加速部分 3.45%） [25]	Web 10%、 Feed1 13%、 Feed2 9%、 Cache 4%	最大的一行；按边界拆分见第 21 页
压缩 / 解压	≈5–6% （约 1/4 的 tax）	2.9% [34]；关键服务 10–50%	Feed1 15%、 Cache1 15%、 Web 11%	95% 字节用弱算法 / 低级别以省 CPU
拷贝（memcpy / memmove）	4–5% （未含内联）	—	内存函数中的最大项	多数拷贝 <512 B
内存分配 / 释放	≈6–7%	4.3% [35] （top-5 应用 3.6–10.1%）	内存函数合计最高 37%； free() 意外地贵	98% 分配 <1 KB， 46% 寿命 <1 ms
哈希	≈1–2%	—	—	—
加密	—	只有服务列那一格： <0.2% [02] （连接复用 >99%）	Cache1 6% （叶子函数）	新建 TLS 连接 1.6 ms / 14 KB [21]
边界专属操作：只在一条边界出现				
通信：网络栈 + RPC 库	≈3–4% （“RPC”项： 1/3 库逻辑 + 2/3 载荷搬运）	RPC 库 1.1% + TCP 栈 1.7% = 2.8% [02]	RPC 收发 I/O： Cache2 52%、 Cache1 38%、 Web 21%	只属于服务边界；不等于 RPC 税（见第 21 页）
日志 / 线程池	—	—	日志： Web 23%；线程池管理 4–11%	日志属于存储边界，学术界很少测
内核（不算可加速的税）	≈18–20% （scheduler ≈5%）	—	缓存类：切换最多 18% [36]，约 30% 时间在内核 [05]	内核 IPC 低且跨代不改善
合计	六项合计 22–27% （叶子函数归因，偏低估）	各项来自不同论文，不能加总	核心应用逻辑最少只占 18% （Web）	DCPerf [05] 转述为 tax 18–82%



Google 一年内 tax 六项占全部周期 22–27% （[01] Fig. 4）

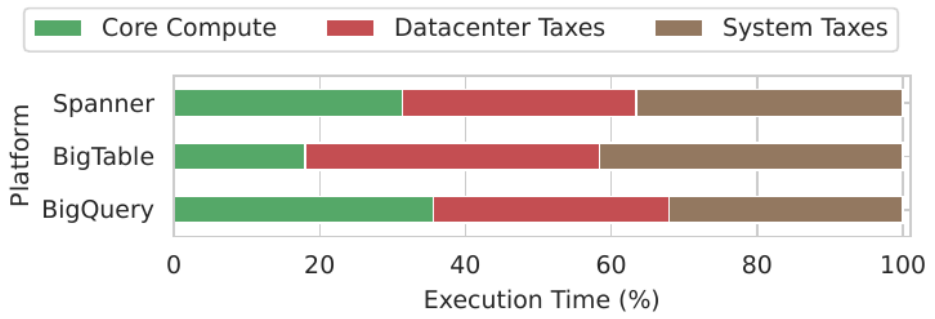


Meta 7 个生产微服务：应用逻辑 vs 编排开销（[20] Fig. 1）

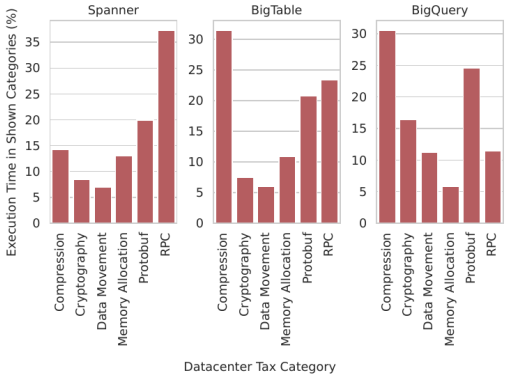
本页只放行合计（这种操作在全 fleet 的总和）。 RPC 路径上的那一份——序列化 1.2%、 TCP 栈 1.7%、 RPC 库 1.1%、压缩——是“服务那一列”的数，放到第二部分对账（第 21 页）。

数据来源与口径： Google 2015 [01] 为叶子函数归因（偏低估）； 2021–24 各项来自不同论文（[35] 分配、 [34] 压缩、 [25] protobuf、 [02] 通信与加密）且口径不同，不能与 2015 直接相减。两个可信趋势：分配下降是 TCMalloc 软件优化的结果；序列化是最大一行。 Meta 2020 [20] 是 7 个重要微服务，不是 fleet 平均。

存储 / 大数据平台的税更重： Spanner 、 BigTable 、 BigQuery 的 CPU 周期里核心计算只占 18–36%



Google 三个生产平台的周期三分：核心计算 / datacenter tax / system tax （[41] Fig. 3 ）



datacenter tax 内部： protobuf 、压缩、 RPC 是三大项（[41] Fig. 5 ）

52%

端到端时间在远程工作（22%） + 存储 IO （30%）

存储 / 分析负载天然是 RPC 密集的 [41]

32–40%

CPU 周期是 datacenter tax

核心计算只有 18–36%； 其余是 system tax [41]

32–42%

system tax： OS 、文件系统、标准库等

OS 18–28% ，标准库最高 53% [41]

20–25%

tax 内部 protobuf 占比；压缩占 14–31%

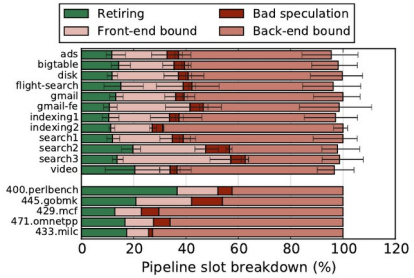
RPC 占 tax 23–37% （数据库平台） [41]

为什么重要：第 9 页的表是 fleet 平均或在线微服务口径；这一页是存储 / 分析侧的画像。两边合起来才是“通用负载”的全貌。
存储和 RPC 是嵌套的：远程存储的每次读写本身就是一次 RPC ，但 RPC 搬的是存储层已经编码、压缩好的块——编码和压缩记在“存储”那一列（第 21 页）。

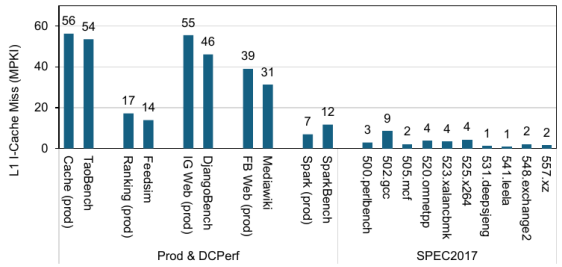
含义：序列化和压缩的加速方案必须能被存储引擎 / 文件格式库调用，只挂在网络 RPC 路径上会漏掉约 65–84% 的编解码操作（第 23 页）。

数据来源与口径： [41] Google ISCA'23 ， Spanner / BigTable / BigQuery 三个生产平台的实流量刻画；百分比为各平台 CPU 周期归类（不是 fleet 平均）。

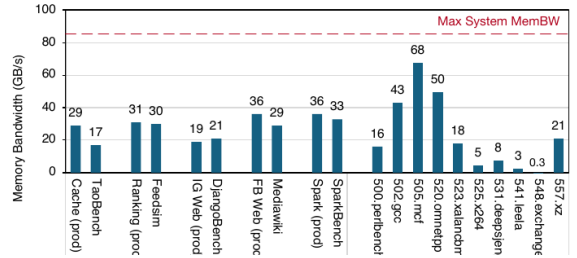
第三种切法：同一批周期按流水线状态看，钱花在前端取指、内核和上下文切换上；传统微服务的内存带宽有富余



Google 12 个应用 vs SPEC 的 Top-Down 分解：前端 stall 明显更高（[01] Fig. 6）



Meta 生产负载 L1-I MPKI 7–56，SPEC 只有 1–9（[05] Fig. 8）



Meta 生产内存带宽 19–36 GB/s，约为系统上限的 30%（[05] Fig. 7）

① 前端是第一瓶颈

Front-end bound 占流水线 slot 的 15–30%（SPEC 的 2–3 倍），5–10% 周期取不到指令 [01]。这是 slot 口径，不与按周期归因的税排序。L1-I 只有 1/4 的 ARM SKU 在 Web 上 Perf/W 低 85% [05]。

② 聪明的预取无效，缺的是容量

Pythia、I-SPY 对单体 +16–19%，对微服务 ≈ 0 [36]。细粒度请求函数工作集只有 0.5 MB；大代码量生产服务的 i-cache miss 来自服务间和 OS 切换的足迹 → 要更大的取指路径、BTB、ITLB。

③ 带宽有富余（传统微服务）

带宽利用率中位 10%、95 分位 31% [01]；生产 19–36 GB/s ≈ 系统 30% [05]；少数热页贡献大部分带宽 [40]。Agent 负载先撞的是内存容量（第 52 页）。

④ 内核占比高

Google fleet 全部内核约 1/5（18–20%）[01]；Meta 缓存类约 30% 时间在内核 [05]；内核 IPC 低且跨代不改善 [20]；384 核上 kernel 6.4 vs 6.9 差 54% [05]。

⑤ IPC 低：SMT 有吞吐空间

生产 IPC 1.0–2.6 [05]，叶子函数用不到峰值一半 [20] → SMT 能提吞吐。但有尾延迟代价：DSec 生产测得兄弟线程混部 +45.2%，core scheduling 后 +17.3% [R10]。

⑥ chiplet / NUCA 是新变量

跨 LLC 域的核间传输延迟是域内的 2.07×，迫使分配器分片 [35]；跨核负载均衡让内核网络栈每阶段慢 2–3× [32]。

⑦ 对 CPI 敏感、对 LLC miss 不敏感

RPC 栈重的 KV 服务尾延迟随 CPI 波动；分支预测失败率和 LLC miss 率与 RPC 延迟不相关 [02]。

⑧ 主频杠杆与小核

Uber：同微架构主频 +9% → P50 −15%、尾延迟 − 1.5× [07]；顺序小核不可用于关键路径 [06]；μManycore 证明小核的问题在软件调度与切换 [36]。

因果：跨边界越频繁，代码足迹越大、切换越多，就表现为前端停顿和内核占比高（Cerebros 卸掉 RPC 层后 i-cache MPKI −93%，模拟 [13]）。对芯片：更大的取指路径与 BTB / ITLB、更便宜的切换与唤醒，比堆内存通道更对症（第 56 页）。

数据来源与口径：[01] Google ISCA'15（Top-Down，Ivy Bridge）；[05] Meta ISCA'25 DCPerf（生产 SKU2，2024）；[36] μManycore ISCA'23 与 SoftSKU ISCA'19；[20] Meta ASPLOS'20；[35] Google ASPLOS'24；[32] Homa/Linux ATC'21；[02] Google SOSP'23；[07] Uber ATC'22；[06] DeathStarBench；[R10] DSec arXiv 2609.22978；[13] Cerebros MICRO'21（模拟）。

成因：一次请求经历了什么（按边界看）

业务怎么拆：三家的业务形态 → 一次请求的一生 → 跨越次数（调用图）→ 每次跨越长什么样 → 形态系数（部署形态 A / B / C ）→ 对账：两本账对得上吗 → 小结
留给下一部分的问题：每个因子能削多少？削的是哪几格？

三家 hyperscaler 的业务形态：都是“多层服务 + 服务间全走 RPC + 存储分离”，差别在语言和部署方式——这决定了跨越次数和形态系数

Google [01][02][17][25]

服务： Search / Gmail / Maps / YouTube 及其后端 Spanner、BigQuery、Bigtable、F1、GFS。

架构： 自称“面向服务架构而非严格微服务”——服务不拆到最小粒度，单个 RPC 常常计算量大。

RPC： Stubby / gRPC + protobuf，几乎全部走 TCP；RPC 产生 95%+ 的应用流量，其中约 75% 与存储系统交互。

语言： C++ 为主（protobuf 周期 88% 在 C++），另有 Java / Python / Go。

运行： Borg 容器；RPC 栈是进程内 library，其下是用户态 Snap，正演化为 Falcon 硬件传输。

Meta [03][05][20][22]

服务： 通用服务器上数千个服务按功耗归 5 类（DCPerf）：Web（HHVM / PHP、Django）、Ranking（C++）、Data caching（TAO / Memcached）、Big data（Spark / Presto，Java）、Media（视频转码）。

硬件： 非 AI 计算只有一种服务器 SKU（单 CPU + 256 GB DRAM）；存储、AI 服务器另算。

RPC： Thrift 二进制 RPC，16 种语言，按代码量 Hack 最多；99% 的 RPC 走进程内 SRLib，1% 走代理。

运行： 产品代码主要是 serverless 函数（FaaS，>50 万台服务器、>1 万工程师），多个函数在同一 Linux 进程里并发执行以提高硬件效率。

阿里 [04][39]

服务： 在线微服务以 Java 为主；2022 年 64,760 个服务 / 187 万实例，中位每服务 4 个实例。

运行： K8s 容器 + 裸金属，与离线批处理混部；在线容器 CPU 利用率常 <10%；DB / Memcached 独立集群。

通信： 无状态服务间 RPC 76%、消息队列 MQ 23%、IPC 1%——MQ 占比明显高于 Google / Meta。

调用图： 按调用次数，5% 的“超级服务”处理 95% 的调用；top 1% 服务 = 87% 调用；常见深度 3。

共性：分布式多层服务，服务间几乎全走 RPC，请求 / 响应用统一格式序列化。差异：语言（C++ / Hack+C++ / Java）与运行形态（library / FaaS 同进程 / 容器混部）——这些差异决定后面每个税项的量级。

数据来源与口径：[01] Google ISCA'15；[02] Google SOSP'23（1 万个 RPC 方法）；[17] Google SIGCOMM'22 流量口径；[25] Google MICRO'21 protobuf 语言分布；[05] Meta ISCA'25 DCPerf；[20][22] Meta ASPLOS'20 / CACM'25；[04][39] 阿里 SoCC'21 / arXiv'25 生产 trace。

一次请求的一生：每一步都在跨边界，每一步做的操作都能在税表里找到对应的行——这是两本账互相印证的第一层



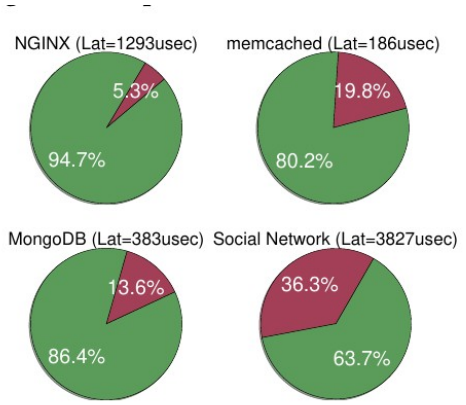
底色 = 跨哪条边界：黄 = 服务边界（RPC） 绿 = 存储边界 红 = 信任边界（体现为形态系数）

步骤	跨哪条边界	做了哪些操作（→ 第 9 页税表的哪一行）	实测量级（口径）	主讲页
① 入口：负载均衡与网关	信任 → 形态系数	终结 TLS → 加密	新建 TLS 连接 1.6 ms、14 KB [21]；连接复用后加密不到 0.2%（Google fleet） [02]	18
② 前端服务	服务 × N	反序列化请求、业务逻辑、扇出 → 序列化、分配	Meta 一次 Fetch 中位 498 个服务块，调用树中位宽 472 [03]	15
③ 每次下游调用（客户端）	服务（+ sidecar 时多两次内核穿越）	序列化 → 压缩 → RPC 库（路由、负载均衡、观测）→ 内核 TCP（拷贝）→ 网卡 → 序列化、压缩、通信、拷贝	Google fleet：TCP 栈 1.7%、RPC 库 1.1%、序列化 1.2% [02]；sidecar 每 RPC 的 CPU 2.7–3.7 倍 [21][18]	22、19
④ 网络	服务	网线与交换机排队（延迟，不耗主机周期）	平均延迟税里网络占 1.1 个百分点（Google） [02]	22、29
⑤ 服务端	服务 + 内核	网卡 → 内核 → 接收队列 → 唤醒线程（切换）→ 反序列化（建对象）→ 调度、通信、分配	排队占 RPC 延迟税 21% [02]；缓存类服务上下文切换最多占 18% CPU 时间 [36]	28、29
⑥ 叶子：存储与缓存	存储	行编码、块压缩、校验哈希、读写盘 → 序列化、压缩、哈希	叶子全是数据库 [03]；存储平台 tax 占平台周期 32–40% [41]；49% 的压缩周期由文件格式库发起 [34]	10、21
⑦ 回程	服务	序列化响应，重复 ③–⑤	—	—
⑧ 旁路：日志、trace、指标、MQ	存储 / 服务	编码日志、拷贝 → 序列化、拷贝	日志占 Meta Web 服务周期 23% [20]；阿里服务间通信 23% 走消息队列 [04]	9、13

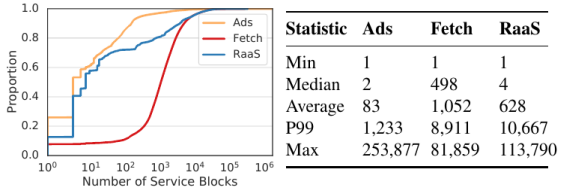
反过来也成立：第 9 页税表的每一行都能在这条路径上找到发生的位置。路径告诉我们“为什么调这么多次”（跨越次数 × 形态系数）；数字能不能对上，见第 21 页对账。

数据来源与口径：路径与步骤划分综合 [02] Fig. 9 的 RPC 延迟分段模型与 [03][04][21] 的生产架构描述，是本报告的示意；各格数字的口径见括号与主讲页。 [21] Meta OSDI'23； [02] Google SOSP'23； [03] Meta ATC'23； [18] mRPC； [36] μManycore； [41] Google ISCA'23； [34] CDPU ISCA'23； [20] Accelerometer； [04] 阿里 SoCC'21 。

跨越次数：拆分即税——单层服务的网络处理占 5–20%，拆成微服务的 Social Network 占 36%；生产系统一次请求中位数百次服务调用



单层服务 5.3–19.8% vs 微服务 Social Network 36.3%（[06] Fig. 3）



(a) CDF

Statistic	Ads	Fetch	RaaS
Min	1	1	1
Median	2	498	4
Average	83	1,052	628
P99	1,233	8,911	10,667
Max	253,877	81,859	113,790

(b) Statistics

Meta 三类请求的 trace 规模：Fetch 中位 498 个服务块（[03] Fig. 10）

公司	服务规模	每请求 RPC 数	调用图形状	来源
Meta	18,500 服务 / 1,200 万实例（去噪后 7,400 / 1,120 万）；复制中位 8、最大 55.7 万	Fetch 中位 498 个服务块（13 个唯一服务）， P99 宽约 7,400； Ads 中位 2	浅（中位深 4）而宽（中位 472、P99 约 7,400）；叶子全是 DB	[03]
阿里 2021	约 2 万服务	常见深度 3； >10% 的图 >40 个服务；大图一半是 Memcached	树状、入度 1； 5% 超级服务处理 95% 调用	[04]
阿里 2022	64,760 服务 / 187 万实例；中位 4 实例	—	密度比 Meta 稀疏 10× 但局部度分布相似； top 1% 服务 = 87% 调用	[39]
Google	1 万 + RPC 方法	半数方法中位后代 ⩽ 13； P99 >1,155	宽 > 深；深度 P99 <10	[02]
Uber	4,000 服务 / 40,000 端点	单 trace 数千个 RPC	深且异步；关键路径只是子集（一个端点 68% 的 P50 延迟来自单个下游）	[07]

Dagger 在同一 benchmark 上：通信占各微服务延迟平均 40%，轻逻辑层达 80%； RPC 处理占各微服务执行时间 5–75%，高负载下占端到端尾延迟 >30% [12][06]。

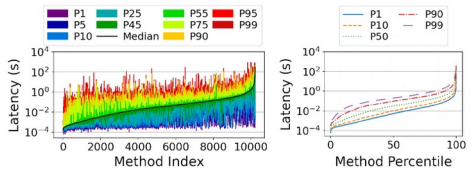
一个修正：对延迟而言只有关键路径上的 RPC 算数（CRISP：一个端点 68% 的 P50 延迟来自单个下游 [07]）；对 CPU 而言全部 RPC 都算数。

减少跨越次数的软件手段（杠杆一，第 27 页）：阿里 7.8% 的依赖是环形、 2.7% 强耦合——合并接口可把远程调用变本地 [04]； Google Service Weaver 让运行时决定同进程还是跨进程 [40]； Meta FaaS 同进程跑多个函数 [22]； CXL 共享内存让同机架调用退化为内存操作 [14][29]。

三个跨公司共性：(a) 宽不深——扇出到分片存储是 RPC 数量的主要放大器；(b) 按调用次数集中——少数超级服务 / 方法承担 87–95% 的调用（不等于周期集中）；(c) 按叶子数，叶子是存储——服务端主体是 DB / 缓存前端。

数据来源与口径：[06] DeathStarBench ASPLOS'19（学术 benchmark，单机）；[03] Meta ATC'23（Ads / Fetch / RaaS 三类生产 trace）；[04][39] 阿里 SoCC'21 / arXiv'25；[02] Google SOSP'23；[07] Uber ATC'22 CRISP；[12] Dagger ASPLOS'21。

每次跨越长什么样（上）：毫秒级为主；中位消息不到 1.5 KB 、尾部几百 KB—— 中位看固定开销，尾部看每字节开销；按调用次数集中不等于按周期集中



按方法的 RPC 延迟分布（[02] Fig. 2 ）

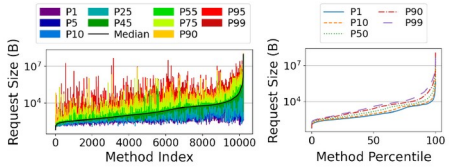
时间尺度

数据

90% 服务的中位延迟 ⩾ 10.7 ms ； 99.5% 的方法 P99 ⩾1 ms ；最慢的 1,000 个方法只占 1.1% 调用但占 89% 的总 RPC 时间。

对硬件设计的含义

学术界 “ µs 级 RPC” 的假设只对 KV / 缓存成立；在 fleet 里，省 CPU 比省 µs 更重要。



按方法的请求大小分布（[02] Fig. 6 ）

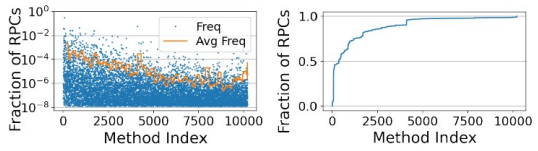
大小

数据

最小 64 B ；一半方法的中位请求 <1.5 KB 、响应 <315 B ；P90 11.8 / 10 KB ； P99 196 / 563 KB 。 Homa 统计的 Google / Facebook 消息 >85% 小于 1,000 B [08] 。

对硬件设计的含义

中位小到单 MTU 、单 cache line 级，单价由每次的固定开销主导；尾部重到几百 KB ，由每字节开销主导——卸载方案两头都要处理。



按方法的调用频率与累计分布（[02] Fig. 3 ）

流行度

数据

top-10 方法占 58% 调用、 top-100 占 91% ； Network Disk 的 Write 一个方法占 28% ；该服务占 35% 调用但 <2% 周期。

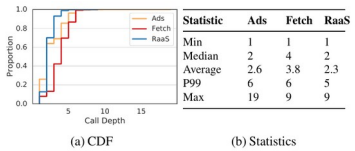
对硬件设计的含义

按调用次数集中，按 CPU 周期并不集中（ Network Disk 35% 调用、 <2% 周期）。少数方法的专用优化能覆盖大部分调用；存储 RPC “ 量大、单个便宜” ，适合 NIC 侧卸载。

一句话： fleet 里的 RPC 是“毫秒级、小消息为主但重尾、按调用次数集中” ——与 µs 级 KV benchmark 的画像相反，硬件目标应先对准 CPU 周期而非往返延迟。

数据来源与口径： [02] Google SOSP'23 ： 1 万个 RPC 方法、 7,220 亿条 Dapper 采样 trace ，图中每个点是一个方法，颜色为该方法的分位数； [08] Homa SIGCOMM'18 引用的 Google / Facebook 消息日志。

每次跨越长什么样（下）：调用树宽而不深、叶子几乎全是数据库、每次的 CPU 成本与大小和延迟都不相关

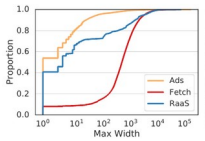


(a) CDF

Statistic	Ads	Fetch	RaaS
Min	1	1	1
Median	2	4	2
Average	2.6	3.8	2.3
P99	6	6	5
Max	19	9	9

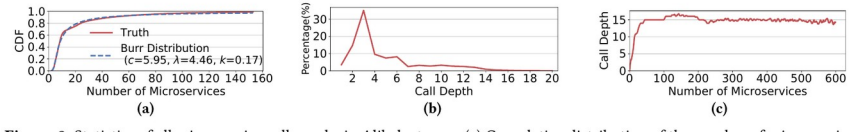
(b) Statistics

Meta：调用深度中位 2–4 （[03] Fig. 11）

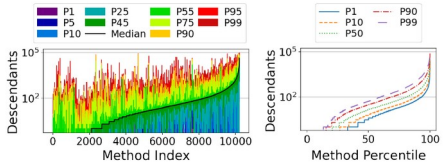


Statistic	Ads	Fetch	RaaS
Min	1	1	1
Median	1	472	3
Average	52	916	507
P99	774	7,395	7,885
Max	166,171	46,746	72,196

Meta：调用树最大宽度的分布（[03] Fig. 12）



阿里 2 万服务：调用图规模 / 深度分布 / 深度 vs 服务数（[04] Fig. 3）



Google：按方法的后代数分布，重尾（[02] Fig. 4）

调用树 · 数据

深度：Google P99 <10 [02]，Meta 中位 4 [03]，阿里常见 3 [04]。宽度：Meta 中位 472、P99 约 7,400 [03]。跨越次数的完整数字见第 15 页（主讲页）。

含义

深度小 → 串行 RTT 累积有限，往返延迟不是主矛盾；宽度大 → 扇出 / incast 和叶子（存储）负载是主体。

叶子 · 数据

Meta：叶子和单中继的服务执行占 53–73%，叶子全是数据库 [03]。阿里：叶子 61% 是有状态服务；大图里一半是 Memcached [04][39]。

含义

按调用次数和叶子数，服务端主体是存储 / 缓存前端——最同质、物理上最集中（独立机群）；按 CPU 周期，大头在毫秒级重服务（第 16 页）。

CPU 成本 · 增速 · 错误

每方法 CPU 成本重尾：P99 比中位高 1–2 个数量级；RPC 大小、延迟都与 CPU 成本不相关。RPS/CPU 每年 +30%。1.9% 的 RPC 出错，“Cancelled” 占 45% 错误、55% 浪费周期（对冲请求）[02]。

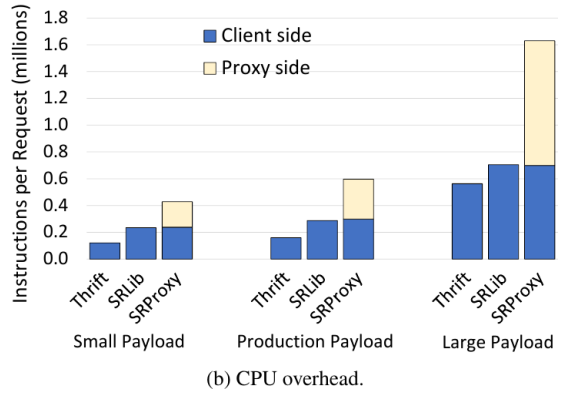
含义

不能按大小预测成本 → 负载均衡只能动态 / 启发式；RPC 增长快于算力；尾延迟优化手段（对冲）本身有 CPU 代价。

数据来源与口径：[02] Google SOSP'23；[03] Meta ATC'23（Ads / Fetch / RaaS 三类 trace，18,500 服务 / 1,200 万实例）；[04] 阿里 SoCC'21（约 2 万服务，注意其公开 trace 的拓扑一致性问题 [40]）；[39] 阿里 2022 trace。

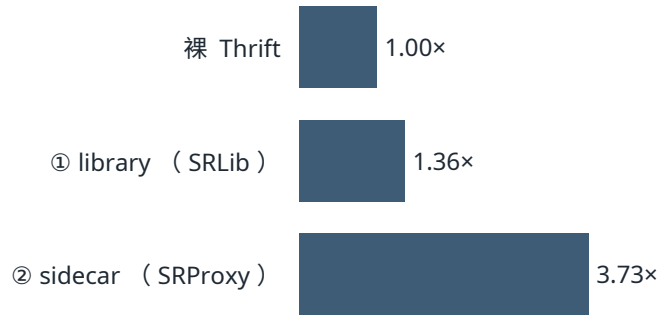
RPC 栈跑在哪（部署形态 A）决定每 RPC 成本：以进程内 library 为基线， sidecar 让每 RPC 的 CPU 成本到 3.7 倍，硬件卸载让传输层 CPU 接近零

形态	成本落在哪块硅	实测系数（基线 = 进程内 library 直连、裸传输）	来源
① 进程内 library （Stubby / SRLib / gRPC-in-app）	应用核 + 内核网络栈	生产聚合 +36% CPU （最坏微基准 +80%） vs 裸 Thrift；延迟 ≈ 裸 Thrift	[21]
② sidecar / 本地代理（Envoy / Istio，SRProxy）	额外的代理核 + 两次额外内核穿越 + 编解码 ×3	CPU +273%、延迟 +107% （Meta）； gRPC+Envoy 中位延迟 63→203 µs、 P99 +180%、带宽 −44%；应用延迟 +27–269%、 vCPU +42–163%；集群 10% CPU / 内存，极端 3× 于应用（阿里云生产）；吞吐 −56%、延迟 2.27× （XLB）	[21][18][23][24][39]
③ 每节点 / 集中式代理（Ambient、Canal 网关、 SRRemoteProxy）	共享代理核（本节点或云网关）	Ambient 比 Istio 代理数 −43%； Canal 比 Istio CPU −12–19×、比 Ambient −4.6–7.2×；但多一跳远程	[24]
④ RPC 作为节点级系统服务（Snap / Pony Express、 mRPC）	独占 / 共享的服务核 + 共享内存跳	Snap： Gbps/ 核 比内核栈 3×，但比独占栈低 3.5–7×， 100 Gbps 需 7–14 核； mRPC 比 gRPC+Envoy 快 6.1× 但比 eRPC 慢 2× （+5 µs IPC）	[38][18][32]
⑤ 内核旁路 library （eRPC、Junction）	应用核（独占轮询或 Junction 式共享）	2.3 µs、 10 Mrps/ 核（100 Gbps 内核栈需 18 核， eRPC 约 1–2 核）； Junction 密度 19–62×	[31][32][40]
⑥ 内核内 L7 （XLB eBPF）	应用所在核，无跨进程	比 Istio / Cilium 吞吐 +1.5×、延迟 − 60%；生产延迟 − 41%、密度 +30%	[39]
⑦ NIC / 加速器卸载（Dagger、Cerebros、 RpcNIC、 Falcon）	NIC / IPU / 专用单元	传输层 CPU ≈0 （Falcon）； RPC 层 CPU −74% （RpcNIC， PCIe）到 − 37–64× （Cerebros，片上模拟）	[30][28][13]



Meta 生产：每请求指令数， Thrift / SRLib / SRProxy （客户端 + 代理侧）（[21] Fig. 10b）

每 RPC CPU 成本倍数（Meta 生产，裸 Thrift = 1）

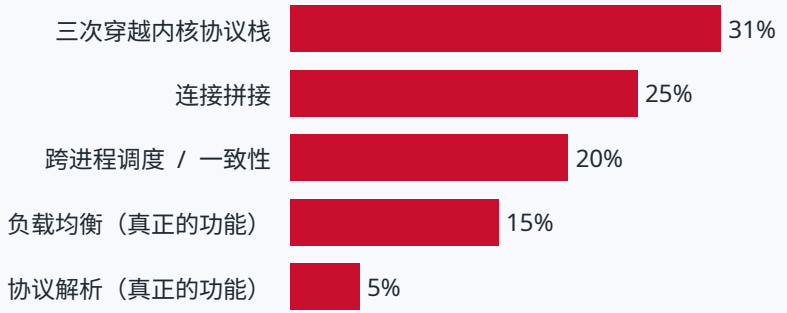


演化方向的共同点： sidecar （每 pod）→ 每节点代理 → 集中网关 → 内核内 / 硬件——每一步都把 L7 处理集中到更少、更专业的位置，与硬件卸载同向。

数据来源与口径： [21] Meta OSDI'23 ServiceRouter （生产聚合）； [18] mRPC NSDI'23； [23] MeshInsight SoCC'23 （benchmark）； [24] 阿里云 SIGCOMM'24 Canal （生产）； [39] XLB arXiv'26； [38] Snap SOSP'19； [32] Homa/Linux ATC'21； [31] eRPC NSDI'19； [40] Junction NSDI'24； [30] Falcon SIGCOMM'25； [28] RpcNIC HPCA'25； [13] Cerebros MICRO'21 （模拟）。

sidecar 为什么贵：三份独立分解一致—— L7 语义本身不贵，贵的是多出来的进程边界、协议栈穿越、重复编解码和拷贝

XLB 对 sidecar 开销的分解 [39]： LB 功能只占 20%



ServiceRouter [21]： 代理开销 74% 在用户态（序列化、加密、身份），零拷贝也救不了。

MeshInsight [23]： HTTP / gRPC 模式下协议解析占 63–77%、TCP 模式下 IPC + socket 写占主导；零拷贝 socket 写对 <4 KB 消息几乎无效。

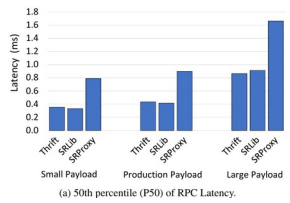
各家怎么选：形态由“信任边界”决定，不是性能

Google / Meta 内部默认 ① library。 Meta 99% 的 RPC 走 SRLib，1% 走代理（只用于 Erlang 等无法链接的运行时）；全量改代理要多 O(100,000) 台机器 [21][22]。Google 的 Stubby 是 library，其下是 ④ Snap 用户态服务，正演化为 ⑦ Falcon 硬件 [38][30]。

hyperscaler 的 7.1% RPC 税（[02] 口径）是 library 形态下的数字，换成 sidecar 只会更高。

公有云租户以 ② sidecar 为主（CNCF： 90% 组织在用或评估 mesh [23]），因为不能改应用、多语言、需要强制策略；阿里云运营 Istio 四年后转向 ③ 集中网关 [24]；业界 2024–2026 的方向是 ③ Ambient / Cilium 与 ⑥ eBPF [24][39]。80–95% 的 mesh 用户需要 L7 功能 [24]，所以纯 L3/L4 的 eBPF 不够。

library 形态不能服务外部客户（无隔离、要改应用） [39]—— 形态选择本质是信任边界决定的。



Meta： P50 延迟 Thrift ≈ SRLib， SRProxy +107% （[21] Fig. 10a）

L7 处理位置的演化：集中到更少、更专业的位置

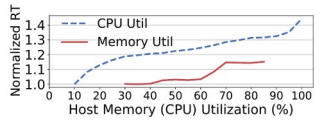
sidecar（每 pod）	每节点代理	集中网关	内核内 L7	NIC / DPU 卸载
Istio： 集群 10% CPU / 内存 [24]	Ambient： 代理数 − 43% [24]	Canal： CPU 比 Istio −12–19× [24]	XLB eBPF： 生产延迟 − 41% [39]	L7 元数据随包交付（待验证）

综合：贵的不是 L7 逻辑，而是为了放置 L7 逻辑多出来的进程边界、协议栈穿越、重复编解码和拷贝——这也是为什么零拷贝对小消息几乎无效，而“减少跳数”有效。

数据来源与口径： [39] XLB arXiv'26 （sidecar 开销分解）； [21] Meta OSDI'23 （生产）； [23] MeshInsight SoCC'23 （benchmark + CNCF 调查）； [24] 阿里云 SIGCOMM'24 （生产运营数据）； [22] Meta CACM'25； [38] Snap； [30] Falcon。

放置、拓扑与隔离环境（形态 B / C ）： 混部干扰、 NUMA 、 CXL 机架和 serverless 都会改变每 RPC 的成本和延迟

形态 C： 放置与拓扑



阿里：主机 CPU 利用率 10→30% ，在线服务 RT 劣化 20% （[04] Fig. 14 ）

有状态服务独立部署：阿里 DB / Memcached 独立集群 [04]； Meta 存储服务器是单独 SKU [05] → RPC 的叶子物理上集中在存储机群，加速器最容易切入。

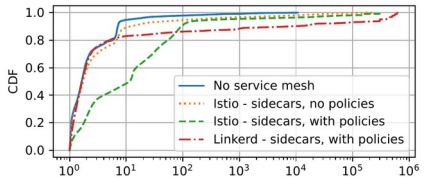
混部干扰：阿里在线容器 CPU <10% ，但主机利用率 10→30% 就让 RT 劣化 20% ，内存利用率 <60% 无影响 [04]； 一个节点平均托管 15 个服务的实例 [39] → 核 / LLC / 带宽的硬件隔离能力决定混部收益。

区域内 vs 跨区域： Meta 区域内跨 DC 无性能惩罚（ fabric 无超订） [22]； Google 跨集群 RPC 的网络延迟就是光速延迟（不是拥塞），可优化的是数据局部性 [02] 。

CXL 机架： HydraRPC 真硬件 1.47 µs [14] 、 RPCool 1.5 µs （ RDMA 回退 17 µs ） [29] → 同 CXL 域内 RPC 成本降一个数量级；但 RPCool 在 DeathStarBench 上与 Thrift 持平—— DB 更新主导时通信优化无效。

NUMA / NIC 位置： NIC 远端 NUMA 节点吞吐 / 核 − 20% [33]； Snap 把引擎绑到 NIC 所在节点——零拷贝下 NIC 局部性 > 应用局部性 [38] 。

形态 B： VM / 容器 / serverless



serverless + mesh 策略：每次调用慢 7×（p50 ）到 254× （p99 ）（[43] Fig. 1 ）

容器 + 裸金属是主流：阿里 K8s 裸金属 [04] 、 Meta Twine 容器 + 本地 SSD [22] 。

Meta 的 FaaS 常驻 >50 万台、无冷启动、同进程多函数—— hyperscaler 独有的效率手段 [22]； 公有云每函数一 VM 的隔离代价高（ Lambda 上用 S3 传状态延迟显著更高） [06] 。

serverless × sidecar 的乘积： Azure Functions 生产 trace 在 Knative 上加 Istio / Linkerd 常见策略（ mTLS 、授权、认证、限流）后 p50 慢 7× 、 p95 38.3× 、 p99 254× ， 90% 调用受影响，每节点多耗 2.5–6.5 核；不加策略时中位只 +6% 但 p99 仍 55×；冷启动 1.2 s → 4 s ；最贵三项 = mTLS 、授权、请求认证 [43] 。注意这是延迟倍数，不能和 sidecar 的 CPU 成本倍数（ 2.7–3.7× ）直接比；成因是函数短、策略重、沙箱与策略高频增删。

IOMMU 税：虚拟化必开的 IOMMU 让网络栈吞吐 / 核 − 26% （每页 map / unmap ，接收端内存管理 30% 周期） [33] 。

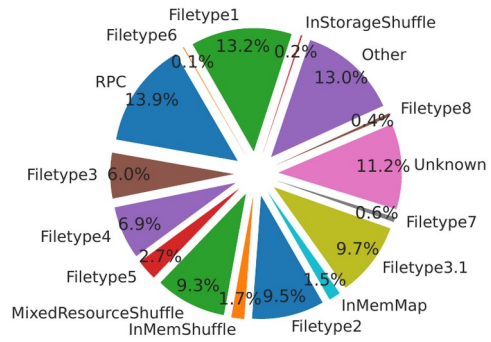
两条链分开看： CPU 成本 library 1.36× 、 sidecar 2.7–3.7× ；延迟 sidecar p50 +107% ， serverless 加策略 p50 7× 、 p99 254× 。每往外一层隔离，成本和延迟都上一个台阶；到 Agent 时代变成“隔离形态系数”（第 50 页）。

数据来源与口径： [04] 阿里 SoCC'21 （生产）； [39] 阿里 2022 trace ； [05][22] Meta ； [02] Google ； [14] HydraRPC ATC'24 （真 CXL FPGA ）； [29] RPCool arXiv'24 （ NUMA 模拟）； [33] SIGCOMM'21 内核栈刻画； [38] Snap ； [43] ETH SoCC'25 （ Azure Functions 生产 trace 在 Knative 上重放）； [06] DeathStarBench 。

对账矩阵（Google fleet）：把两本账放进同一张表——行合计是数据中心税，服务那一列是 RPC 税，两者只在黄色几格上重叠

操作 ↓ ／边界 →	服务（RPC）	存储（含日志）	进程内与其他	行合计
序列化（protobuf）	1.2 [02]；用 [25] 推算得 0.8	其余约 8.4： Bigtable / Spanner 的行、日志、进程内传递，未再细分 [25]	（与左格合计）	9.6 [25]
压缩	**约 0.4： RPC 库直接发起的占压缩周期 13.9%（推算）⚠ [02] 记为 3.1**	约 1.4：文件格式库发起 49% [34]	约 1.1： shuffle 约 13%，其他与未知约 24% [34]	2.9 [34]
通信专属（网络栈 + RPC 库）	2.8： TCP 栈 1.7 + RPC 库 1.1 [02]	远程存储的读写也走 RPC，已计入左格	—	2.8
加密	<0.2 [02]	空	空	空
拷贝	一部分：接收端拷贝占单条长流 >50% 周期（测试床） [33]	空	空	4–5 （2015） [01]
内存分配	一部分：反序列化时建对象	空	空	4.3 [35]
哈希	空	空	空	1–2 （2015） [01]
调度与切换	一部分：排队占 RPC 延迟税 21% [02]	—	—	scheduler ≈5（2015，内核内） [01]
列合计	**[02] 口径 7.1；压缩只计 RPC 库发起的部分则约 4–5（推算）**	存储平台的 tax 占平台周期 32–40%（平台口径，不是 fleet） [41]	空	2015 年六项 22–27 [01]

单位：占 Google fleet 全部 CPU 周期的百分比。各格来自不同论文、不同年份，只对量级。“空” = 公开文献没有按边界拆分的数据。



Google fleet 压缩周期按“谁直接调用”分（[34] Fig. 4）

压缩这一格怎么算的： RPC 13.9% × 2.9% ≈ 0.4%；文件格式（Filetype 1–8）合计 49% ≈ 1.4%；各类 shuffle 合计约 13%； Other + Unknown 约 24%。

为什么标 ⚠： [02] 把 3.1% 记进 RPC 税，几乎等于全部压缩；原文只写“用 GWP 识别 RPC 周期”，没说明压缩怎么归类（第 24 页）。

“数据中心税”和“ RPC 税”不是包含关系：前者是行合计，后者是黄色那一列。矩阵里的“空”格，就是第 63 页 950 待测清单要补的数。

数据来源与口径： [02] Google SOSP'23 Fig. 20 （GWP）； [25] ProtoAcc MICRO'21； [34] CDPU ISCA'23 Fig. 4 （压缩按直接调用方的分布）； [33] SIGCOMM'21 主机网络栈（100 Gbps 测试床）； [35] Google ASPLOS'24； [41] Google ISCA'23； [01] Kanev ISCA'15。拆分与“约 4–5%”为本报告推算（OUTLINE_RPC_DECK_V3_LOGIC.md 4.4 节）。

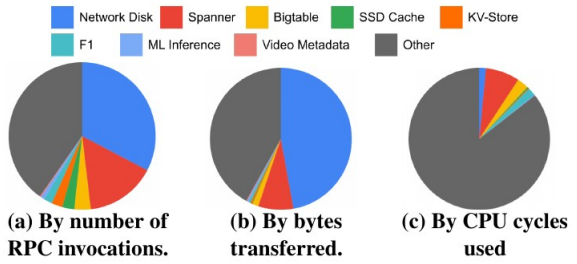
服务边界这一列（RPC 税）要按货币分开讲：字节 95%+、CPU 周期 7.1%（[02] 口径）、延迟平均 2%（尾部 38–96%）

货币一：字节 —— RPC 是 DC 通信的主体

95%+

Google 生产 DC 的应用流量由 RPC 产生

其中约 75% 与存储系统交互 [17]



Google 头部 RPC 服务：按调用数 / 字节 / CPU 周期（[02] Fig. 8）

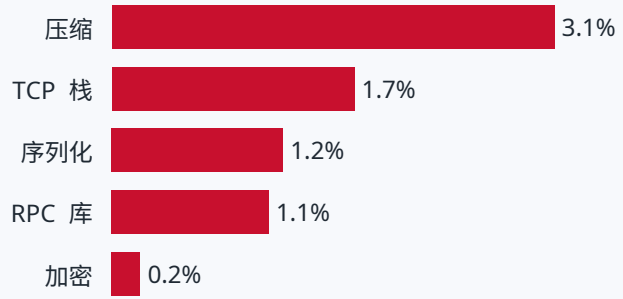
- 存储是主体中的主体：Network Disk 一个服务占 35% 调用但 <2% 周期——“量大、单个便宜”。

货币二：CPU 周期 —— [02] 口径 7.1%

7.1%

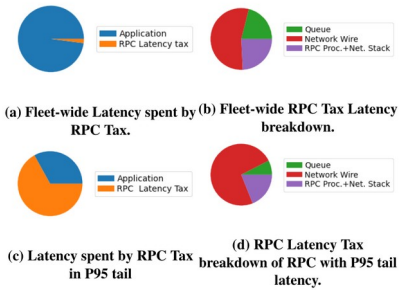
Google fleet 全部 CPU 周期中的 RPC 税

分解（占 fleet 周期 %）[02]：



- 压缩 3.1% 几乎等于全 fleet 压缩 2.9% [34]，而 RPC 库直接发起的只占其中 13.9%—— 严格按 RPC 路径约 4–5%（推算，第 21 页）。
- Meta 头部服务是分布的另一端：RPC 收发 I/O + 序列化 + 压缩合计 Cache2 ≈59%、Web ≈59%、Feed1 ≈28% [20]。

货币三：延迟 —— 平均只占 2%，尾部由税主导



上：fleet 平均 RPC 延迟税 2%；下：P95 尾部里税占比大幅上升（[02] Fig. 10）

平均 2.0%：网络 1.1%、栈 0.49%、排队 0.43%；中位方法 8.6%。

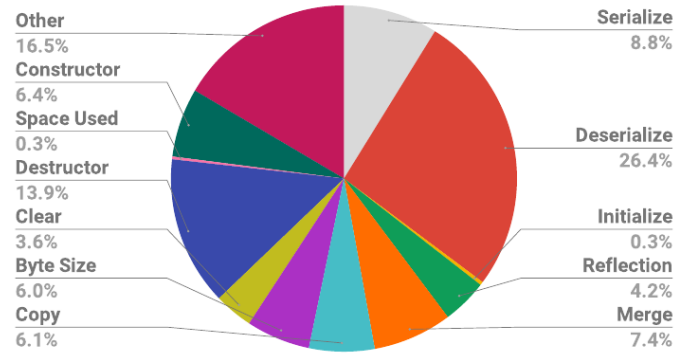
尾部 38% / 96%：开销最高的 10% 方法，中位税 38%、P90 96%；排队占税的 21%。

原因：生产 RPC 是 ms 级的（90% 服务中位延迟 ⩾ 10.7 ms），µs 级的栈开销被应用处理淹没。

字节、周期、延迟是同一列的三种货币，不能互相换算；Google fleet 平均与 Meta 头部服务是同一分布的两端（第 24 页），引用时必须说明口径。

数据来源与口径：[17] Google SIGCOMM'22（2021 年生产流量）；[02] Google SOSP'23，1 万个方法、7,220 亿条 Dapper trace，CPU 归因用 GWP；[34] CDPU ISCA'23；[20] Meta ASPLOS'20，7 个微服务的叶子函数归因。

交叉格：序列化这一行只有约八分之一落在服务列—— protobuf 占 fleet 9.6%， RPC 路径上约 1.2 个百分点；引用同一个数要回到原文口径



Google fleet 的 C++ protobuf 周期按操作分解：反序列化 26% 、序列化 9% （[25] Fig. 2）

protobuf 占 9.6% fleet 周期，其中反序列化 2.2% 、序列化 1.25% 可加速，其余是“胶水代码” [25] 。

但只有 16% 的反序列化、 35% 的序列化来自 RPC 栈，其余来自 Bigtable / Spanner 的行、日志、进程内传递 [25] 。

三角验证： RPC 路径序列化 ≈ 16.3%×2.2% + 35.2%×1.25% ≈ 0.8% ，与 Cloud-Scale 独立测得的 1.2% [02] 同一量级。

两种分母别混： 16% / 35% 是可加速的编解码操作里来自 RPC 栈的比例；按全部 protobuf 周期， RPC 路径约 1.2 个百分点（约八分之一）。“序列化税” 9.6% 与“ RPC 税” 7.1% 只重叠约 1 个百分点。

同一个数的不同说法（本调研反复遇到）

原文数字	被引用成	提醒
Kanev 2015： protobuf ≈3–4% （读图值） [01]	ProtoAcc 引为 5%； Zerializer / Optimus Prime 引为“ RPC+protobuf 12%”； mRPC 引为“ gRPC 库 10%”	引用时回到原文读图
Accelerometer： 序列化按服务 4–13% [20]	zBuffer / Cornflakes 引为“序列化 6.7%”	是 7 个微服务的某种平均，不是 fleet
DCPerf： Meta 各服务 tax 18–82% [05]	常被当作 fleet 平均	原始结论是 Accelerometer“ 应用逻辑最少 18%”[20] ，是 7 个服务的范围
DeathStarBench： 网络处理 36% [06] ； Cerebros： RPC 层 40–90% [13]	常与 Google 7.1% 并列比较	轻业务逻辑 + 已优化传输的设定，比 fleet 高一个数量级；对应“热点服务”而非“ fleet 平均”目标
阿里 2021 公开 trace [04]	大量论文据此做调用图分析	99.48% 的拓扑不一致问题 [40] ，引用其调用图结论需注明
Google RPC 税 7.1% [02]	被当作“ RPC 不重要”的证据	是 library 形态、 fleet 平均、 ms 级服务的数，不能外推到 sidecar 用户和 KV 前端；其中压缩 3.1% 的归类存疑（第 21 页）

读数规矩：先问“谁测的、 fleet 平均还是热点服务、 RPC 路径还是全口径、中位还是 P99”，再比较。

数据来源与口径： [25] Google MICRO'21 fleet GWP 采样； [02] Google SOSP'23 ； [20] Meta ASPLOS'20 ； [05] Meta ISCA'25 ； [06] DeathStarBench ASPLOS'19 （学术 benchmark ）； [13] Cerebros MICRO'21 （模拟）； [40] ICPE'24 对阿里 trace 的一致性检查。

对账结论：两本账有五处对得上、三处对不上——对不上的地方，正是 950 要和客户一起测的东西

对得上的五处

① **序列化 × 服务这一格：** ProtoAcc 推算约 0.8% [25]， Cloud-Scale 直接测得 1.2% [02]，两种独立方法同一量级（第 23 页）。

② **字节多、周期少：** RPC 搬了 95%+ 的字节，周期只占几个百分点。protobuf 按字节 92% 是 bytes / string （已编好的载荷） [25]，压缩一半由文件格式库发起 [34]——RPC 主要在搬存储层编好、压好的块（推断，与 RPC 流量 75% 与存储交互 [17] 一致）。

③ **拆分放大服务列：** 单层服务网络处理 5.3–19.8% → 拆成微服务 36.3% [06]； Meta Fetch 中位 498 个服务块 [03]。

④ **形态系数：** Meta 99% × 1.36 + 1% × 3.7 ≈ 1.38×， fleet 由 library 决定 [21][22]；公有云按 sidecar 系数外推增量 12–19% CPU，阿里云实测 sidecar 占集群 10% CPU [24]，同一量级。

⑤ **两端是同一分布：** Meta 头部服务的服务列占 28–59% [20]，Google fleet 平均 7%； fleet 按周期加权，毫秒级重服务把平均拉低。

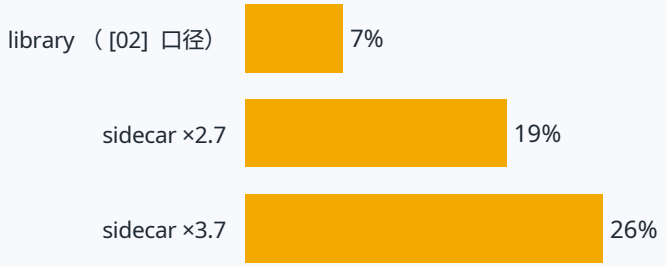
对不上的三处

⚠ **压缩那一格差约 8 倍：** [02] 把 3.1% 记进 RPC 税，几乎等于全 fleet 压缩 2.9% [34]；但 [34] Fig. 4 中由 RPC 库直接发起的只占 13.9% （约 0.4%）。[02] 原文只写“用 GWP 识别 RPC 周期”，没说明压缩怎么归类。本报告两个口径并列： [02] 口径 7.1%；严格按 RPC 路径约 4–5% （推算）。

② **四行没有按边界的拆分：** 拷贝、分配、哈希、调度都没有公开的按边界数据。 v2 的“ RPC 全路径 10–15%”把 2015 年的拷贝和分配叠到 2023 年的 7.1% 上，年份和口径都不一致， v3 删去。

③ **形态占比没有公开：** 各部署形态在 fleet 中的流量和 CPU 占比，任何公司都没有公开——形态系数只能算单价，算不出总账。

形态系数外推 & 数据空白



RPC 税占 CPU 周期（%）： library 为 Google fleet 实测（[02] 口径）， sidecar 两档按 Meta / mRPC 系数外推

数据空白（转入第 63 页）：

- (a) 各形态在 fleet 中的流量与 CPU 占比
- (b) 拷贝、分配、压缩按边界的拆分
- (c) Java / 托管语言的 RPC / 序列化周期
- (d) 网络 ： 存储序列化的比例
- (e) sidecar 卸载到 DPU 后的真实 CPU 节省

读法：对得上的五处说明“税是跨边界的成本”这套框架站得住；对不上的三处和矩阵里的空格，合起来就是 950 待测清单里传统负载那一半。

数据来源与口径：外推与推算为本报告计算（ OUTLINE_RPC_DECK_V3_LOGIC.md 4.4 节； REPORT.md 2.5 节）； [02] Google SOSP'23； [25] ProtoAcc； [34] CDPU ISCA'23 Fig. 4； [17] Google SIGCOMM'22； [06] DeathStarBench； [03] Meta ATC'23； [21][22] Meta； [24] 阿里云 SIGCOMM'24； [20] Accelerometer； sidecar 系数 2.7–3.7 取自 [21][18]。

负载洞察小结：现象（按行）和成因（按列）合起来的八条结论

1 按周期没有杀手应用，按调用高度集中

CPU 周期： top-1 程序约 10%、 top-50 约 60% 且逐年下降，应用内部同样平坦。调用次数： top-100 方法占 91%。调用集中不等于周期集中。

2 税的组成（行合计， Google fleet）

序列化 9.6% > 内存分配 4.3% ≈ 拷贝 4–5% （2015） > 压缩 2.9% > 通信专属 2.8% （TCP 栈 1.7% + RPC 库 1.1%）；全部内核约 18–20% 不算可加速的税。各项年份不同，只看量级。

3 税在核上的样子

前端停顿占 slot 15–30%、生产 L1-I MPKI 7–56；内核约 1/5；缺的是取指容量而非更聪明的预取；传统微服务带宽利用率 10–31%； SMT 有吞吐空间也有尾延迟代价。

4 税是跨边界的成本，拆得越细越重

请求路径上每一步的操作都能在税表里找到对应的行；同样的业务，拆成微服务后网络处理从 5–20% 升到 36% （DeathStarBench）。

5 跨越次数多、每次跨越小而重尾

Meta Fetch 中位 498 个服务块，调用树宽不深，叶子是存储； RPC 毫秒级为主，中位 <1.5 KB （固定开销主导）， P99 几百 KB （每字节开销主导）。

6 形态系数：成本和延迟两条链

CPU 成本： library 1.36× → sidecar 2.7–3.7×；延迟： sidecar p50 +107% → serverless 加策略 p50 7×、 p99 254×。形态由信任边界决定。

7 RPC 税是服务那一列

[02] 口径 7.1%，压缩只计 RPC 库发起的部分则约 4–5% （推算）；序列化 9.6% 只有约 1.2 个百分点在这一列；存储那一列更重（Spanner 等平台 tax 32–40%）。

8 对不上的地方就是数据空白

压缩的归类、拷贝 / 分配按边界的拆分、形态在 fleet 中的占比、托管语言的序列化税——公开文献都没有，芯片厂商应与客户联合测量。

对硬件目标的含义：先对准 CPU 周期里的整行（序列化、压缩、拷贝合计十几个百分点）与前端 / 切换效率，其次才是往返 µs；按调用次数，最同质的落点是存储 / 缓存前端；增量市场是 sidecar / serverless 用户。

数据来源与口径：各条结论的原始数据与口径见第 8–24 页页脚与附录 B （第 65 页）；数字均为论文原文或本报告的显式推算。

三

削减：四个杠杆（学界怎么削）

四个杠杆与放置规则（吃行还是吃列）→ 通信栈的成本机理 → 五条路线 → 硬件卸载谱系 → 序列化的成本结构 → 序列化与压缩加速器放在哪

四个杠杆：减次数、降系数、降每次固定开销、降每字节开销——硬件放在哪，决定它吃到一整行还是只吃一列

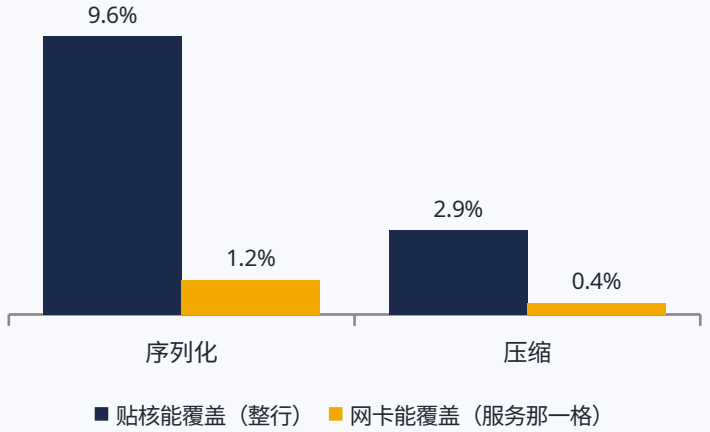
杠杆	改公式的哪一项	代表方案（主讲页）	落在哪	吃行还是列
① 减次数	跨越次数	合并接口（阿里 7.8% 的依赖是环形 [04]）； Service Weaver 由运行时决定同进程还是跨进程 [40]； Meta FaaS 同进程跑多个函数 [22]； CXL 共享内存让同机架调用退化成访存 [14][29]（第 15 、 29 页）	软件； CXL / UB 的内存语义	整列变小
② 降系数	形态系数	sidecar → 每节点代理 → 集中网关 → 内核内 eBPF [24][39]； mRPC 把策略搬进节点级服务 [18]（第 18 、 19 页）	软件（mesh 团队）； DPU 网关	整列变小
③ 降每次固定开销	单价里的固定部分	内核旁路 eRPC / Snap [31][38]； 硬件分发 RPCValet [11]； 硬件调度 nanoPU / μManycore [10][36]； 硬件传输 Falcon [30]； 过载控制 Breakwater [16]（第 28–30 页）	网卡 / UB / 片上 NI； 核的切换与唤醒	吃列
④ 降每字节开销	单价里的每字节部分	零拷贝 Cornflakes / zBuffer [26][19]； 贴核 ProtoAcc [25]； 近核 CDPU [34]； DSA 拷贝引擎（第 31 、 32 页）	贴核 / 近核 KAE	吃行

五条路线（第 29 页）落在哪个杠杆：传输协议重设计、服务器端调度 → ③，同时改延迟；内核旁路 → ③；硬件卸载 → ③ + ④，但只吃服务那一列； CXL 共享内存 → ①，并顺带降单价。

中位看固定开销，尾部看每字节：中位 RPC <1.5 KB 、 93% 的 protobuf 消息 ≤ 512 B [02][25]，片外卸载的盈亏点约 425 B [20]—— 小消息上零拷贝几乎无效，出路是减次数和批量；几百 KB 的尾部才是④的主场。

同一个改进，收益取决于服务类型： Meta Fetch 中位 498 个服务块 × 每次省 5 μs CPU ≈ 每请求 2.5 ms [03]； 对 P95 SLO 500 ms 的排序服务是 0.5%，对毫秒级 KV 是 100%+（本报告粗算）。

放置规则：贴核吃整行，网卡只吃一格



占 Google fleet 周期 %： 行合计 [25][34]； 服务那一格 [02]，压缩为推算（第 21 页）

贴核（核内指令、近核 KAE）：不管为哪条边界做的都能用。

网卡 / UB / DPU：只吃走网络的那一份。

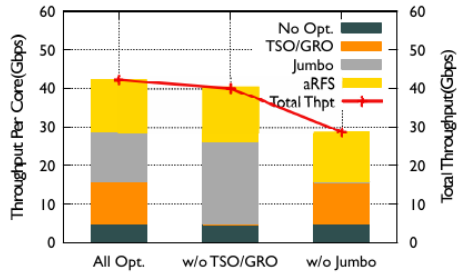
软件：改次数和系数，不改单价。

用处：第 32 页的“放置之争”和第 55 页的“税 × 硅”都从这条规则来。

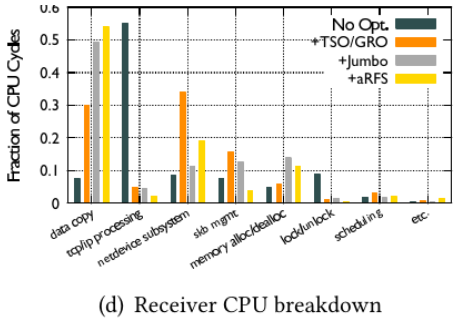
①② 改的是“跨越次数 × 形态系数”，③④ 改的是“每次单价”；硬件放在哪，决定它吃一整行还是只吃一列。后面五页按这个框架读，每页右上角标出所属杠杆。

数据来源与口径：杠杆划分为本报告的组织方式（OUTLINE_RPC_DECK_V3_LOGIC.md 4.5 节）；方案与数字见各主讲页： [04] 阿里； [40] Service Weaver / Junction； [22] Meta； [14][29] CXL RPC； [24][39][18] mesh； [31][38] [11][10][36][30][16] 通信栈； [26][19][25][34] 序列化与压缩； [20] Accelerometer 盈亏点； [03] Meta ATC'23 。

通信栈的成本机理：主机侧网络成本 = 拷贝 + 跨核分发 + cache 争用 + 地址翻译，而不是协议计算



单长流：全部优化开启也只有约 42 Gbps/ 核（[33] Fig. 3a ）



(d) Receiver CPU breakdown

接收端 CPU 分解：数据拷贝 >50% （[33] Fig. 3d ）

单核已跑不满线速。 100 Gbps 下 Linux 内核栈最优约 42 Gbps/ 核（jumbo + TSO/GRO + aRFS 全开）；接收端内核→用户的数据拷贝占单长流 >50% 周期； 400 Gbps 下 payload 拷贝 >60% CPU 、仅 memcpy 就需 ⩾ 15 核 [33] 。

协议逻辑不是主要成本。 Homa 做进 Linux 内核后拥塞被协议消除，但短消息往返软件开销约 9.5 µs （> 网络 5.6 µs ），分散在各组件、无单一热点；跨核负载均衡（ RSS → NAPI → SoftIRQ 两级哈希）使每阶段慢 2–3× ； 100 Gbps 双向 80% 需 ⩾ 18 核 [32] 。

LLC 是被忽视的 I/O 瓶颈。 DDIO 只用 2 个 LLC way ， BDP 增长超过 LLC 增长 → 单流 49% cache miss 、吞吐 / 核 − 24% ； 200 Gbps × 1 KB 包 = 每 41.8 ns 一次 I/O ，一次 miss 就破线速； CEIO 用“总信用 = LLC 容量”实现零 miss ，吞吐 +1.2–2.9× [33] 。

其他系数： IOMMU −26% ； NIC 远端 NUMA −20% ；长短流混跑同核 − 43% ；拥塞控制算法的选择对吞吐 / 核无影响——瓶颈在接收端 [33] 。

TCP 的六宗罪（ Ousterhout ）： 连接开销、字节流 HOL 、公平调度抬高尾延迟、发送端驱动 CC 保证高负载下有排队、不用网内优先级、按序投递限制负载均衡；且实现太复杂难以硬件化 [32] 。

四项成本分别对应四个硬件设计空间（文献的需求见第 35 页）

拷贝

DMA 引擎 / 零拷贝、 SG 小段开销

跨核分发

NIC 按实时核负载动态分发

cache 争用

LLC 信用 / 分区、 NIC 可见的占用反馈

地址翻译

IOMMU / ATS 、大页 DMA 、更大 IOTLB

对硬件的含义：主机侧网络成本 = 拷贝 + 跨核分发 + cache 争用 + 地址翻译，而非协议计算——所以“更快的 TCP 卸载引擎”不是答案，数据路径和缓存路径才是。

数据来源与口径： [33] SIGCOMM'21 Host Network Stack Overheads （ 100 Gbps 测试床， Linux 5.4 ）及 CEIO SIGCOMM'25 / FlexiNS arXiv'25 的 400 Gbps 数据； [32] Homa/Linux ATC'21 （ 40 节点集群， 25/100 Gbps ）。

杠杆①③④ · 硬件卸载只吃列

通信方向的五条路线：传输协议重设计、用户态 / 内核旁路、服务器端调度与过载控制、硬件卸载、CXL 共享内存

① 传输协议重设计：消息语义、接收端驱动、网内优先级

Homa （SIGCOMM'18）：真实 RPC 消息 >85% <1 KB；盲发首个 RTTbytes + 8 级交换机优先级近似 SRPT + 接收端逐包授权，短消息 P99 比 TCP 低 100× [08]。 Aequitas （Google 生产）： RPC 大小与业务优先级不相关，按 RPC 标 QoS + 发送端 AIMD 准入， fleet P99 −10%，无需新硬件 [17]。 Falcon （Google IPU）： 200 Gbps、 120 Mops/s、拥塞下比 RoCE 好 8×、 IP 只占 NIC 面积 5–6% [30]。

② 用户态与内核旁路：软件能做到多好

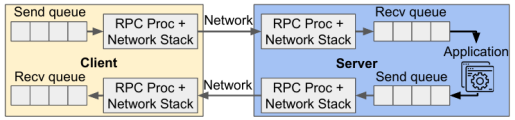
eRPC 是软件下界： 2.3 µs、 10 Mrps/ 核、有损以太网不靠 PFC，但独占轮询核、无隔离 [31]。 Snap （Google 生产）：栈作为用户态服务，首要动机是迭代速度，代价是 100 Gbps 双向需 7–14 核 [38]。 mRPC： 序列化 + 策略搬进节点级服务，比 gRPC+Envoy 快 6.1× 但比 eRPC 慢 2× （+5 µs IPC） [18]。 Junction： 解决内核旁路的密度问题，每机多跑 19–62× 实例 [40]。

③ 服务器端调度、分发与过载控制

Google： 排队占 RPC 延迟税 21%， >50 µs 的调度唤醒与尾延迟强相关 [02]。 RPCValet： RSS 静态分流浪费 30–76% 吞吐， NI 侧“replenish 即空闲”推送分发距最优 3–16% [11]。 nanoPU： NIC 直达寄存器文件 69 ns、硬件线程调度 <1 ns 决策，尾延迟 − 5× [10]。 µManycore： 1024 小核 + 硬件队列 / 切换，尾延迟 − 10.4× [36]。 Breakwater： 服务端信用准入 <20 ms 收敛 [16]。

⑤ CXL 共享内存 RPC （④ 硬件卸载见下页）

HydraRPC （真 CXL 硬件） 1.47 µs、大消息比 RDMA RPC 快 96–247×，主动绕开 CXL.cache 用非缓存共享 [14]； RPCool 1.5 µs （RDMA 回退 17 µs），只有 ⩾ 2 页的参数 seal 才比 memcpy 划算， DeathStarBench 上与 Thrift 持平 [29]。限制： CXL 域内（机架）、同 ABI、通知只能轮询。



Google 的 RPC 延迟分段模型：发送队列 → RPC 处理 + 网络栈 → 网线 → 接收队列 → 应用 → 回程（[02] Fig. 9）

表 3-8 通信方向：方案 × 削减项 × 成熟度

削减的成本项	软件方案	硬件方案	成熟度
网络排队 / 尾延迟（网线段）	Homa、 Aequitas （生产）	Falcon （量产 IPU）、 nanoPU 硬件 CC	生产
内核栈 CPU （TCP 1.7%、拷贝）	eRPC、 Snap （生产）、 Junction、 Homa/Linux	Falcon、 FlexiNS、 CEIO （LLC 信用）	生产 / 原型
服务器排队与分发（接收队列段）	R2P2 JBSQ、 Breakwater、 Protego	RPCValet / NeBuLa / nanoPU 硬件分发与调度、 µManycore	原型 / 模拟
RPC 库 + 分发（1.1%）	mRPC	Cerebros、 Dagger、 RpcNIC	原型
跨进程 / sidecar 跳数	mRPC、 XLB （eBPF）、 Canal 集中网关	DPU L7 （待验证）	部分生产
通信 + 拷贝 + 序列化（CXL 域内）	HydraRPC、 RPCool	CXL 控制器功能	早期

五条路线的共同模式： 面向消息 + 无连接；接收端驱动的信用 / 授权（Homa 的 grant、 Breakwater 的信用、 R2P2 的 JBSQ、 Aequitas 的准入分别控制链路、服务器 CPU、网络优先级）；把需要全局状态的功能（CC、 pacer、分发）下沉到 NIC / 硬件——这一跳只能靠硬件消除 [32]。

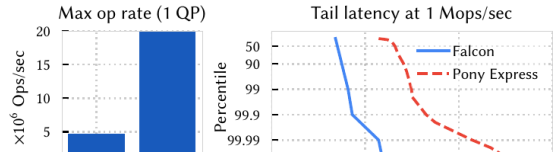
Google 的取舍： Falcon 明确否定接收端驱动路线（假设网络无超订）和包喷洒（要求路径同构），保留面向连接但用内存带宽 + 二级缓存压平 >100K 连接的悬崖 [30]。

数据来源与口径： [02] Google SOSP'23； [08] Homa SIGCOMM'18； [17] Aequitas SIGCOMM'22 （Google 50 集群生产）； [30] Falcon SIGCOMM'25 （IPU E2100 实测）； [31] eRPC NSDI'19； [38] Snap SOSP'19； [18] mRPC； [40] Junction / R2P2 / Protego / NeBuLa； [11] RPCValet ASPLOS'19 （模拟）； [10] nanoPU OSDI'21 （FPGA）； [36] µManycore （模拟）； [16] Breakwater； [14] HydraRPC ATC'24； [29] RPCool arXiv'24。

杠杆③④ · 吃列（服务边界）

硬件卸载谱系（FPGA NIC → 片上 NI → IPU）：接口语义比物理带宽更重要、小任务必须由 NIC 自主触发、 DPU 的 ARM 核不是加速器

方案	卸载内容	CPU– 设备接口	关键结果	主要限制
DeathStarBench FPGA [06]	TCP 栈（bump-in-the-wire）	PCIe	网络处理延迟 – 10–68×，端到端尾延迟 – 43%–2.2×	只卸传输，未报告 CPU 节省
Dagger （ASPLOS'21） [12]	整个 RPC 栈（接口 / 序列化 / 分发 / 传输）	UPI 内存互连（类 CXL）：CPU 只写共享 buffer	每核 12–16 Mrps、 2.1 µs；内存互连比 doorbell +14%； memcached 延迟比内核低 11.4×	MTU = 1 cache line；FPGA 回环
Cerebros （MICRO'21） [13]	Thrift RPC 层三模块（头 / 载荷 / 分发）	片上 NIC （NeBuLa）	RPC 层周期 – 37–64×，总周期 – 1.8–14×； i-cache MPKI –93%；面积比每核复制少 5×	全模拟； RPC 层 40–90% 是轻逻辑设定
nanoPU （OSDI'21） [10]	传输 + 核选择 + 调度 + 数据路径	寄存器文件	69 ns 线到线； 118 Mrps/ 核； +15% 核逻辑	只对 <1 µs cache 常驻请求
RpcNIC / RPCAcc （HPCA'25） [28]	RPC 层 + protobuf 序列化	PCIe SmartNIC （U280）	序列化 – 2.3× / –4.3×；主机 CPU –74%； RX 与片上持平、 TX 差 1.24–1.4×	主机需预序列化（22% 周期）； 250 MHz FPGA
FlexiNS （arXiv'25） [33]	可编程传输（BF3 ARM 只做包头）	PCIe， header-only TX	400 GbE 线速、单流接近 RDMA NIC	ARM 内存 / 链路带宽是硬约束
Falcon （SIGCOMM'25） [30]	可靠传输全部（CC / 重传 / 多路径 / 整形 / 加密 / 隔离）	PCIe （IPU E2100 ASIC 同 die IP），verbs / NVMe 兼容	200 Gbps、 120 Mops/s、单 QP 20 Mops、 1 B–2 MB 5.8–8.7 µs 且 P99 ≈ 中位；面积 5–6% / 功耗 3–4%	片上 O(BDP) 缓冲 1–2 mm²； RPC ULP 未评估



Falcon 硬件 vs Pony Express 软件：操作率 5×、尾延迟 10×（[30] Fig. 1）

规律 1 接口语义比物理带宽更重要。 Dagger 同一链路从 doorbell/DMA 换成缓存一致性共享内存就 +14%，并消除 MMIO 和批处理需求 [12]——CXL.mem/.cache 接入 NIC 的论据；但 CXL 物理层仍是 PCIe， sub-µs 对指针追踪不够 [28]。

规律 2 PCIe 的两个陷阱及解法： 小 DMA 打满事务率 → 设备侧 SRAM 攒一条消息一次性写（+1.78–2.2×）；指针追踪跨 PCIe → 主机先拉平、设备只编码；代价是 CPU 仍参与，节省上限约 74–78% [28]。

规律 3 卸载粒度决定成败： 片外盈亏点 425 B [20]、 Cerebros 每次卸载约 200 周期 [13]、 93% protobuf 消息 ≤ 512 B [25]—— 小而频繁的任务必须由数据到达方（NIC）自主触发，不能由 CPU 逐次调用。

规律 4 SoC DPU 的 ARM 核不是加速器。 nanoPU （延迟至少 2×）、 RpcNIC （编码慢 5.9×）、 FlexiNS （内存带宽 500 vs 需 1,600 Gbps）、 BenchBF3 （DPA 单线程更弱）四方一致 [10][28][33][40]。

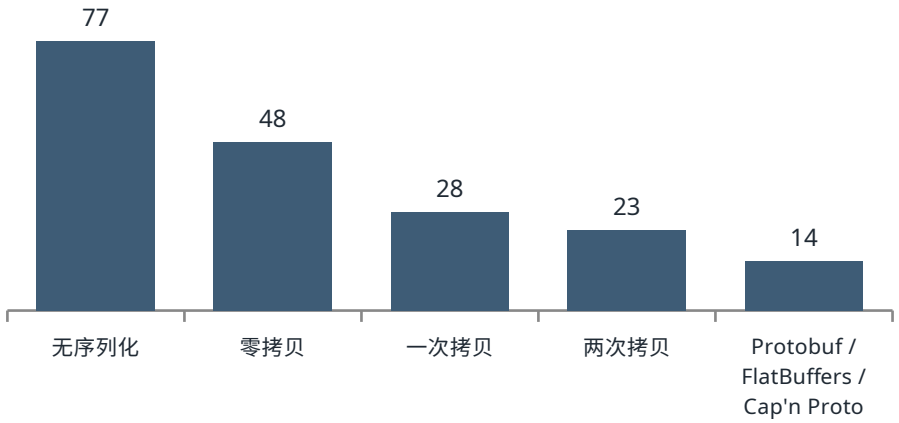
DPU 的现实（2024–25 刻画）： BF-3 DPA 单线程算力 / 访存远弱于主机核、 on-path RDMA 延迟比 off-path 高 12–30%、 DMA 引擎轮询模式某些大小比 BF-2 慢 21–39% [40] → 正确用法是“ARM 决定、硬件搬运”（header-only 卸载 + 主机内存直通 SG），而不是把 RPC 栈整体搬到 ARM 上跑。

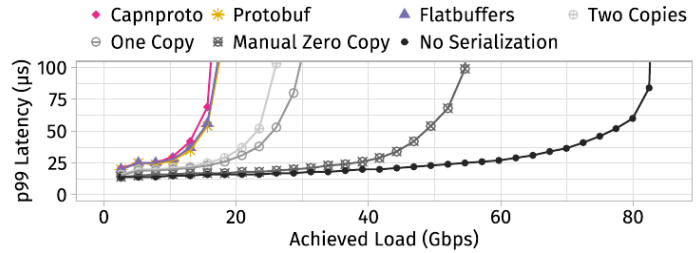
两条路线： Falcon （ASIC 流水线 + ARM 只算慢变量：单 N1 核 20 M events/s 足够） vs BF-3 （ARM / DPA 做数据面）——本调研的证据一边倒支持前者；这直接约束了 UB Controller 与 DPU 的产品定义（第六部分，第 60 页）。

数据来源与口径：表中数字为各论文自报； Cerebros、 µManycore 为模拟， Dagger / nanoPU / RpcNIC 为 FPGA 原型， Falcon 为量产 IPU 实测； [40] BenchBF3 ICNP'24 与 BlueField Idiosyncrasies ICS'25； [20] Accelerometer； [25] ProtoAcc。

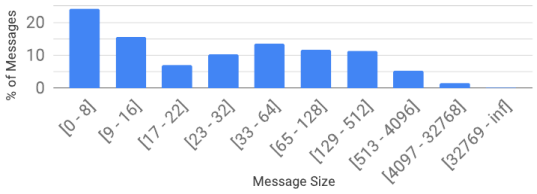
序列化的成本结构：拷贝是主体、元数据其次、编码最少——硬件若只做编码（varint 指令）或只做 DMA，都只能拿到一小块

单核 echo 服务的吞吐（Gbps）：每多一次拷贝掉一档，主流库只剩 1/5（Cornflakes [26]）

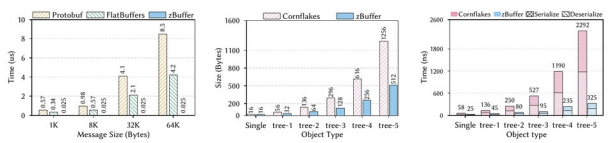




同一实验的 p99 延迟 vs 负载曲线（[26] Fig. 2）



Google fleet protobuf 消息大小：93% ≤512 B （[25] Fig. 3）



阿里 zBuffer：去元数据 + 零拷贝后编解码开销比 Protobuf 低 1–2 个数量级（[19] Fig. 9）

fleet 级画像（Google，ProtoAcc [25]）： protobuf 占 9.6% fleet 周期（C++ 88%）；96% 字节仍是 proto2 → 格式长期稳定；消息极小（24% ≤8 B、56% ≤32 B、93% ≤512 B），但 >32 KB 的 0.08% 消息承载 ≥ 13.7× 于最小桶的字节；按字节 92% 是 bytes / string；小 varint 每字节比大字段慢 100–500×——"没有银弹"；90% 消息只填了 <52% 的字段。

成本分解三份独立实验一致： Cornflakes 单核 echo 77 → 48 → 28 → 23 → 13–15 Gbps [26]；zBuffer：Protobuf 13–14 Gbps → 去元数据 23.4 → 再零拷贝 45 Gbps [19]；zFlatBuffers：缓冲区扩容拷贝占序列化时间 38–58% [40]。Kanev 2015：RPC 税的 2/3 是载荷搬运 [01]；Meta：内存函数最高 37% 周期，memcpy 最大项 [20]。

托管语言更重： Spark 六个应用即使用 Kryo，序列化 / 反序列化仍占平均 28%（最高 83.4%）——对象图递归遍历、反射、GC 头、间接加载叠加 [37]。

反序列化的复杂度在对象构造与内存分配（ProtoAcc 的 arena、Cerebros 的预留缓冲、zBuffer 的就地修指针、Cereal 反序列化器面积 5× 于序列化器）——与 malloc 税 4.3% 是同一问题。

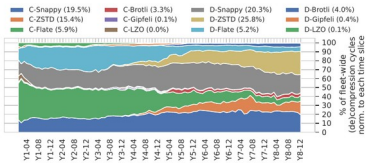
序列化税 = 拷贝 + 元数据 + 编码 + 对象分配。"拷贝是主体"来自吞吐型实验；中位 RPC 不到 1.5 KB，小消息上零拷贝的簿记会吃掉收益（下页），出路是减少跳数和批量。

数据来源与口径：[26] Cornflakes SOSP'23（单核 echo 微基准，100 Gbps CX-5）；[25] Google MICRO'21 fleet GWP；[19] zBuffer PPoPP'26（阿里，已生产部署）；[40] zFlatBuffers IPCCC'23；[01] Kanev；[20] Meta；[37] Cereal ISCA'20（Spark 六应用）。

杠杆④ · 贴核吃行 / NIC 只吃列

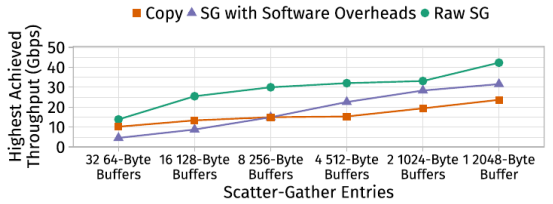
序列化加速器放哪：贴核吃整行（全部 9.6%），NIC 侧只吃服务那一格（约 1.2 个百分点）；PCIe 附接对中位消息是负收益

方案	放置	覆盖格式 / 语言	面积 / 性能	关键论据
ProtoAcc （MICRO'21）[25]	贴核（RoCC，共享 L2 / TLB，虚拟地址）	protobuf proto2， C++	deser 0.133 + ser 0.278 mm²（22 nm）；比 Xeon 快 3.8×；估省 >2.5% fleet	约 84% 的反序列化不在 RPC 路径；93% 消息 ≤ 512 B → PCIe NIC 放置不可行
Optimus Prime （ASPLOS'20）[27]	片上共享（NoC 边缘 tile）	通用 DT， Thrift / protobuf	12 流水线 0.45 mm² / 532 mW（28 nm）；约 60× 于软件	PCIe 约 1 µs 对嵌套消息多次往返 → 否决；收益来自预取重叠访存
Cerebros （MICRO'21）[13]	片上 NIC 侧	Thrift compact， C++	RPC 层 − 37–64×；面积比每核复制少 5×	分发必须与序列化同在硬件； NIC 位置可做亲和核选择
RpcNIC （HPCA'25） [28]	PCIe SmartNIC （协同）	protobuf + 可重构内核	主机 CPU −74%；序列化 − 2.3× / −4.3×	主机预序列化拉平 + 设备侧一次性 DMA 规避 PCIe； DSA 帮拷贝
Cereal （ISCA'20） [37]	内存侧加速器（读写 JVM 堆）	Java 对象图（改格式）	2.7 mm² / 1.2 W （40 nm）；Spark S/D 7.97× / 4.81×	反序列化器面积 5× 于序列化器；需与 GC 协作
软件零拷贝： Cornflakes / zBuffer [26][19]	商用 NIC scatter-gather	C++ / Rust，同 ABI	zBuffer 1 KB 编解码 25 ns （Protobuf 1/22.8）； Masstree P99 −21%、吞吐 +62%	裸 SG 64 B 就赢、加簿记后 ≥ 512 B 才赢 → 混合策略；需要 RDMA / DPDK



Google fleet 压缩周期按算法： Snappy / ZStd / Flate 为主（[34] Fig. 1）

压缩：序列化的姊妹问题，约占 fleet 周期 3%。整行 2.9% [34]；由 RPC 库直接发起的只占 13.9%（约 0.4%，推算）， [02] 记进 RPC 税的 3.1% 归类存疑（第 21 页）；关键服务 10–50%、存储平台 tax 的 14–31% [41]。 95% 字节用弱算法 / 低级别以省 CPU → 加速后可换回容量；中位调用 64–128 KiB；文件格式库发起 49% 的压缩周期 [34]。相对 Xeon 软件：近核 CDPU 10×（0.431 mm²）、 chiplet 9.5×、 PCIe 1.8× [34]。阿里云放弃本地 QAT 改用远程密钥服务器 [24]—— 片外加速器在云上落地失败的生产案例。



裸 SG 64 B 就赢；加上内存安全簿记后 ≥ 512 B 才赢（[26] Fig. 3）

放置决策链：

- 1. PCIe 附接对中位消息是负收益（盈亏点 425 B [20]；比近核慢 3.4–5.6× [28][34]），除非 RpcNIC 式协同（CPU 仍付 22%）。
- 2. 片上三种放置没有绝对赢家：贴核覆盖全部序列化（含存储 / IPC 的 65–84%）且面积极小；共享硅效率高但受 NUCA 影响； NIC 侧能做整栈卸载但只覆盖 RPC 路径（编解码操作的 16–35%，全部 protobuf 周期的约八分之一）。决定因素是客户负载里“网络：存储序列化”的比例——目前没有公开数据。
- 3. chiplet / I/O die 是新的中间选项（CDPU，均相对 Xeon 软件： chiplet 9.5×、近核 10×、 PCIe 1.8×） [34]。
- 4. 格式几乎总要改（Optimus Prime 改 setter、 Cereal 改格式、 zBuffer 改布局）；只有 ProtoAcc 保持 wire 兼容。落地障碍是软件接口和生态，不是硅面积（<1 mm²）。
- 5. 调用模型比放置更重要（存储侧）：片上 vs 片外同步只差 1.04×，异步比同步好 1.3×，加速器链式执行（压缩→加密→哈希不回核）≈ 全异步 → “一组可链式调用的小加速器” [41]。

决定序列化加速器放哪的不是硅面积（<1 mm²），而是客户负载里“网络：存储序列化”的比例和软件接口生态；压缩则已有近核方案的定量边界（10× / 0.43 mm²）。

数据来源与口径： [25] ProtoAcc （RTL + FireSim）； [27] Optimus Prime （模拟）； [13] Cerebros （模拟）； [28] RpcNIC （FPGA）； [37] Cereal （模拟）； [26] Cornflakes （实测）； [19] zBuffer （生产）； [34] CDPU ISCA'23 （Google fleet 画像 + RTL）； [41] Google ISCA'23 极限研究； [24] 阿里云 SIGCOMM'24； [20] Accelerometer； [02] Google SOSP'23。

四

综合：每一格多大、已证明能削多少

热点地图（按矩阵的行排列：行合计 / 服务那一格 / 热点服务 / 落在哪块硅 / 杠杆与已证明的削减）→ 文献对 CPU / NIC / 互连 / SKU 的需求清单 → 按客户类型的差异与争议；取舍与排序统一放在第六部分
留给下一部分的问题： AI 来了，这张表还成立吗?

热点地图：按矩阵的行排列——每一行的总量、落在服务列的那一格、热点服务量级、落在哪块硅、哪个杠杆已证明能削多少

操作（行）	行合计（Google fleet）	服务那一格（RPC 路径）	热点服务量级	落在哪块硅	杠杆 · 已证明的削减（比例）	成熟度
通用操作（行）：贴核的方案吃整行，网卡上的方案只吃服务那一格						
序列化	9.6%，可加速 3.45% [25]	1.2% [02] （≈ 0.8% [25]）	Web 10%、 Feed1 13% [20]； Spark 28% [37]；大数据平台 tax 的 20–25% [41]；轻逻辑微服务 40–90% [13]	应用核	④ 贴核 ProtoAcc 3.8–6.2×、 0.41 mm² [25] （吃行）； zBuffer 编解码约 1/23、吞吐约 3× [19]； NIC 侧 Cerebros 37–64× [13]、 PCIe RpcNIC −74% CPU [28] （吃列）	软件生产（zBuffer）；硬件原型
压缩	2.9% [34]	约 0.4% （推算）； [02] 记为 3.1%	10–50% （存储、排序） [34]； Feed1 / Cache1 15% [20]；大数据平台 tax 的 14–31% [41]	应用核	④ 近核 CDPU 10×、 0.43 mm² （吃行）， PCIe 只有 1.8× [34]；加速后可换回容量	原型（RTL） + 商用 QAT （片外，云上落地差 [24]）
拷贝	4–5% （2015） [01]	一部分（网络栈拷贝）	网络栈拷贝 >50–60% [33]； Ads1 最高 [20]	应用核 + LLC	④ 零拷贝 SG （≥ 512 B） [26]； DSA / I/OAT −55% 主机周期 [28][38]；片上拷贝 +12.7% [20]	DSA 商用
内存分配	4.3% [35]	一部分（反序列化建对象）	Cache1 最重 [20]	应用核 + TLB	软件 TCMalloc fleet +1.4% [35]；硬件卸载只 +1.86% [20]	软件
服务边界的专属操作（整行都在服务列）：网卡 / UB 与核的切换						
通信： RPC 库	1.1% [02]	整行	Cache2 收发 I/O 52% （含部分内核网络） [20]	应用核	③ Cerebros （与序列化一起）； Google： 单独加速价值不高 [02]	模拟
通信：网络栈	1.7% [02]；全部内核 18–20% [01]	整行	Cache 约 30% 时间在内核 [05]； 100 Gbps 双向需 18 核 [32]	应用核 + 内核	③ Snap 3×/ 核 [38]； eRPC 10 Mrps/ 核 [31]； Falcon 传输 ≈ 0 核、 IP 占 NIC 5–6% 面积 [30]； CEIO +1.2–2.9× [33]	Snap / Falcon 生产
调度 / 唤醒 / 分发	scheduler ≈5% （2015，内核内） [01]	排队占 RPC 延迟税 21% [02]	Cache 切换 18% [36]； Linux 切换 5K 周期 [36]	内核 + 核	③ RPCValet +29–76% [11]； nanoPU 尾延迟 − 5× [10]； μManycore −10.4× [36]； Breakwater <20 ms 收敛 [16]	模拟 / FPGA
不是某一行，但改变系数、其他货币或流水线						
形态系数（sidecar）	Meta 只占 1% 流量	每 RPC 2.7–3.7× [21][18]	公有云集群 10% CPU [24]； serverless + 策略 p50 7× / p95 38× [43]	代理核 + 内核	② 集中网关 − 12–19× [24]； eBPF 内核内 +1.5× [39]； mRPC 6.1× [18]	部分生产
延迟：网络尾部	平均延迟税里占 1.1 个百分点 [02]	尾部主导	—	交换机 / NIC	③ Homa 100× [08]； Aequitas fleet −10% [17]； Falcon vs RoCE 8× [30]	生产
流水线：前端 i-cache / ITLB	15–30% slot [01] （slot 口径，不与周期排序）	—	Web、 Cache [36]	核前端	大 L1-I （反例： DCPerf ARM SKU −85% Perf/W） [05]；卸 RPC 层 MPKI −93% [13]	微架构
跨核一致性 / IOMMU	—	网络栈每阶段 2–3× [32]；IOMMU −26% [33]	chiplet 跨域 2.07× [35]	NoC / LLC / IOTLB	小一致性域、 NUCA 感知分配器； ATS / 大页 DMA	模拟 / 微架构

按行合计排序（Google fleet，各项年份不同）： 序列化 9.6% > 分配 4.3% ≈ 拷贝 4–5% （2015） > 压缩 2.9% > 网络栈 1.7% > RPC 库 1.1%；服务那一格加起来就是 RPC 税（[02] 口径 7.1%，严格约 4–5%）；全部内核 18–20% 不算可加速税。

硅位置： 绝大多数开销落在应用核上（所以 Meta 单 SKU、 Perf/W 决定采购）；贴核方案吃整行， NIC 方案只吃服务那一列； NIC / 交换机还承担网络尾延迟。单类服务的大项（缓存服务收发 I/O 52% [20] 等）见第 36 页。

数据来源与口径：“已证明的削减”指现有论文的削减比例；“落在哪块硅”指该开销当前主要消耗哪块硬件；①–④ 为第 27 页的四个杠杆；引用编号见附录。

文献对硬件提出的需求： CPU 侧、 NIC / DPU 侧、互连侧、服务器 / SKU 侧四组；先后取舍放在第六部分

CPU 侧

近核“数据变换加速器簇”：序列化 + 压缩 + memcpy。三者合计十几个百分点； ProtoAcc 0.41 mm² 对应可加速的 3.45%， CDPU 0.43 mm² 对应压缩约 3% 并换回容量；文献要求贴核或片上、异步加链式调用；收益应按 zBuffer 而非 Protobuf 基线算。

片上 DMA / 拷贝引擎的用户态低延迟提交与小段优化。多数拷贝 <512 B，小段固定开销要 <100 ns；与 NIC SG 协同。

前端：大 L1-I / ITLB、切换感知的指令 cache 管理，而非更聪明的顺序预取器；替换策略可调（FB Web +2.9%）。

硬件请求调度与上下文切换：消息驱动的核唤醒、硬件线程队列与优先级抢占（nanoPU −5×、 μManycore −10.4×）。

LLC 作为 NIC↔CPU 缓冲的主动管理：可配置 DDIO way、 NIC→LLC 信用、按队列 cache 分配（CEIO +1.2–2.9×）。

chiplet / NUCA 感知：暴露 cache 域拓扑、可动态划分一致性域（跨域 2.07×）。

IOMMU / ATS：− 26% 的虚拟化税； ATS / PRI、大页 DMA、更大 IOTLB。

内存保护 key 与 CXL 共享内存安全：更多 key、用户态可控的页 / 对象级权限切换（RPCool 0.35 μs 沙箱）。

不值得单独硬件化： malloc （硬件只 +1.86%）、 RPC 库本身、内核——靠软件与微架构通用改进。

NIC / DPU / IPU 侧

消息式硬件传输（Falcon 功能清单）：延迟型 CC + 硬件时间戳 + 接收侧缓冲占用回传、位图 SACK、拥塞感知多路径 + 片上 O(BDP) 缓冲（1–2 mm²）、请求–响应事务接口、不依赖无损网络； 18 核 → ≈ 0 核。

按实时核负载的动态分发（替代 RSS）： RSS 浪费 30–76% 吞吐；需要 NI↔核 ns 级反馈——片上 / 封装内 NI 才能做，是 CPU 厂商的结构性优势。

SG / DMA 小段开销与接收侧 SG：条目数上限 ~30 → 更多、每条目 <100 ns、设备侧按消息聚合写回、接收侧按 schema 分散落点。

RPC 边界与优先级元数据保留： Aequitas 需要按 RPC 而非连接标 QoS。

L7 解析结果作为元数据交付（“L7 RSS”）：解析占 sidecar 开销 63–77% 且应用又解析一遍；但跳数和拷贝才是主体， DPU 搬走代理并不自动省钱。

ARM / DPA 只做控制面与包头， payload 走硬件直通。

加密加速要么标配要么别做：价值在连接建立和 sidecar / 网关形态，需要低延迟小批量路径。

互连侧（CXL）

CXL 对 RPC 的价值在接口语义和机架内共享，不在延迟量级。 Dagger +14%、 HydraRPC 1.47 μs；但 DB 主导的微服务无收益。

需要的控制器 / CPU 功能：廉价的非缓存共享、内存写触发的轻量通知（当前只能轮询）、按主机的访问窗口 / 租约 / 回收、多主机内存保护。

放置策略随之改变：同 CXL 域内服务对用共享内存 RPC （1.5 vs 17 μs）→ 调度器把高频通信服务放进同一域； CXL 挂旧 DRAM 省碳。

服务器 / SKU 侧

单一 SKU 约束 + Perf/W + 碳： Meta 非 AI 单 SKU； DCPerf 靠 Perf/W 选中绝对性能更低的 ARM SKU；“同一芯片、多运行模式”比多 SKU 更现实。

内存：带宽过配、延迟与热页优先。利用率 10–31% → 通道换容量 / CXL 扩展 + 小容量高带宽层。

拓扑： NIC 位置与 NUMA。 NIC 远端 − 20%；多 die 下 NIC 挂载位置影响 RPC 路径。

用真实负载做设计输入并交付内核调优： DCPerf 指导 ~10 项优化 FB Web +38% / Perf/W +47%， SPEC 完全看不出； 384 核内核锁争用 54%。

这是文献视角的需求全集，不是行动清单：哪些由鲲鹏做、先做哪个，在第六部分按“量级 × 鲲鹏差距 × 可行性 × 归属”统一排序（第 62 页）。

数据来源与口径：每条建议的依据数字见 REPORT.md 5.2 节与本 PPT 对应页；

同一个发力点对不同客户价值不同： hyperscaler 、公有云租户、 Java / 大数据、存储 / KV 前端、CXL 机架采用者

客户类型	RPC 形态	RPC 税量级	最相关的发力点
hyperscaler （Google / Meta 型）	library （99%）、 C++ / Hack 、自研传输（Snap → Falcon ）	fleet 7% （RPC 路径） + 序列化全口径 9.6% + 压缩 2.9%	近核序列化 / 压缩簇、前端、 Falcon 类消息传输、动态分发；评估工具 DCPerf / HyperProtoBench
公有云租户 / 企业（Istio 型）	sidecar （每 RPC 的 CPU 2.7–3.7× ）→ Ambient / 集中网关 / eBPF； serverless 加策略（延迟 p50 7× 、 p95 38× ）	集群 10–30% CPU 在 mesh 上； FaaS 每节点 +2.5–6.5 核	L7 元数据交付、加密、集中网关的 DPU / 加速； XLB 类内核内 L7 需要的是核内效率而非卸载
Java / 大数据（阿里、 Spark ）	容器混部、 RPC 76% / MQ 23% 、 JVM	序列化 28% （Spark ）； CPU 干扰敏感	Cereal 类 JVM 对象序列化加速（空白市场）、硬件隔离（核 / LLC / 带宽 QoS ）、快速唤醒（利用率 <10% ）
存储 / KV 前端（按调用次数的服务端主体）	高 QPS 、 µs 级、内核重	收发 I/O 52% [20]； 内核约 30% [05]； 切换最多 18% [36] （不可相加）	硬件调度与切换、 LLC 管理、动态分发、消息式传输； nanoPU / µManycore 路线的目标客户
CXL 机架架构采用者	同机架共享内存 RPC	通信 + 拷贝 + 序列化三项在域内接近消失	CXL 共享内存 RPC 与控制器功能

争议与开放问题（也是芯片厂商可以用第一手数据回答的）

1 加速器瞄 fleet 平均还是热点服务? Google 说 RPC 库加速价值不高、序列化 / 压缩更值得 [02]； Cerebros 说 RPC 层占 40–90% [13] 。两者都对：一个是 fleet 平均，一个是轻逻辑服务。怎么取舍见第六部分优先级页（第 62 页）。

2 序列化加速器放哪? 贴核（覆盖全部 9.6% ） vs NIC 侧（只覆盖约 1.2 个百分点但能整栈）——取决于客户负载的网络 ：存储序列化比例，目前没有公开的 fleet 级数据（第一个空白）。

3 CXL 一致性要不要? HydraRPC 绕开 CXL.cache 并断言其开销不可容忍 [14]； RPCool 假设一致性可用 [29]； Dagger 从一致性互连获益 14% [12] 。真实 CXL 3.x 多主机一致性硬件的实测尚无。

4 DPU 的定位： Falcon （ASIC 流水线 + ARM 控制面） vs BF-3 （ARM / DPA 数据面）；证据一边倒支持前者，但 FlexiNS 证明后者在正确的编程模型下也能到 400 GbE 线速 [33] 。

5 零拷贝对小消息无效（MeshInsight 、 Cornflakes 、 Canal eBPF 小包）——中位 RPC <1.5 KB ，小消息的出路是减少跳数和批量；硬件应提供“消息驱动的批量 DMA”。

6 / 7 两个数据空白：托管语言的 RPC / 序列化税几乎没有 fleet 级数据（只有 Spark 的 28% ）；部署形态的 fleet 占比在任何公司都未公开——三角验证无法闭合的原因。

验证工具： DCPerf （5 类负载 + tax 微基准）、 HyperProtoBench 、 HyperCompressBench 、 DeathStarBench ；收益预测 = Accelerometer （卸载上限） × MeshInsight （按调用图放大） × CRISP （关键路径决定延迟收益）。

数据来源与口径： REPORT.md 5.3–5.5 节；表中量级来自第一、二部分对应页；争议各方引用见括号。

五

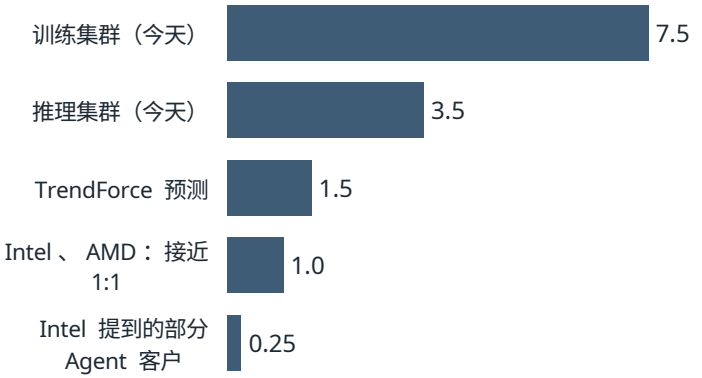
AI 与 Agent：同一张表，多了三列和一种货币

为什么现在要看：市场信号 → 负载单位从请求变成会话 → Agent 在哪里运行、用多少 CPU → 一次 Agent 请求的一生 → 老的行在新列上的格子 → 新列的专属行（五个新税）→ 三个专题对应三条新列 → 小结
留给下一部分的问题：鲲鹏在每一格上做什么？

市场信号（一）：各方都说 Agent 会把 CPU:GPU 从 1:4–1:8 推向 1:1，甚至 CPU 多于 GPU——但说这话的都是卖 CPU 的

谁	什么时候、什么场合	说了什么	身份
Intel （CEO 陈立武、CFO Zinsner）	Q1 2026 财报会（2026-04-23）；JPMorgan 会议（2026-05）	部署比例已从约 1:8 变成 1:4，可能接近 1:1；训练约每颗 CPU 配 7–8 颗 GPU、推理 3–4 颗，Agent 可能翻转为 CPU 多；部分前沿模型客户在 Agent 场景要 4 颗 CPU 配 1 颗 GPU	卖 x86 CPU
AMD （CEO Lisa Su）	Q1 2026 财报会（2026-05-05）	从 1:4–1:8 走向接近 1:1，Agent 多时 CPU 可能多于 GPU；服务器 CPU 市场的 2030 年预测从约 600 亿美元上调到 1,200 亿美元以上（年增 35% 以上），且是新增需求、不挤占 GPU	卖 CPU 和 GPU
NVIDIA （CFO Kress）	Q2 FY27 财报会（2026-08）	Vera CPU 2026-03 起单独销售、已量产；本财年 CPU 收入可见度约 200 亿美元，下一财年翻倍以上；称 CPU 是新的 2,000 亿美元市场；GPU 柜内仍是 1:2	卖 GPU 和 CPU
Arm	2026 年的产品与生态发布	每 GW 需要的 CPU 核从约 3,000 万增到 1.2 亿（4 倍）；自研 AGI CPU 的客户需求超过 20 亿美元（FY27–28）	卖 CPU IP 与芯片
TrendForce	2026-04-14 的分析	今天 1:4–1:8，未来 1:1–1:2，没有给年份	分析机构（付费报告）

每颗 CPU 配几颗 GPU （越往下 CPU 越多）



怎么读这些说法

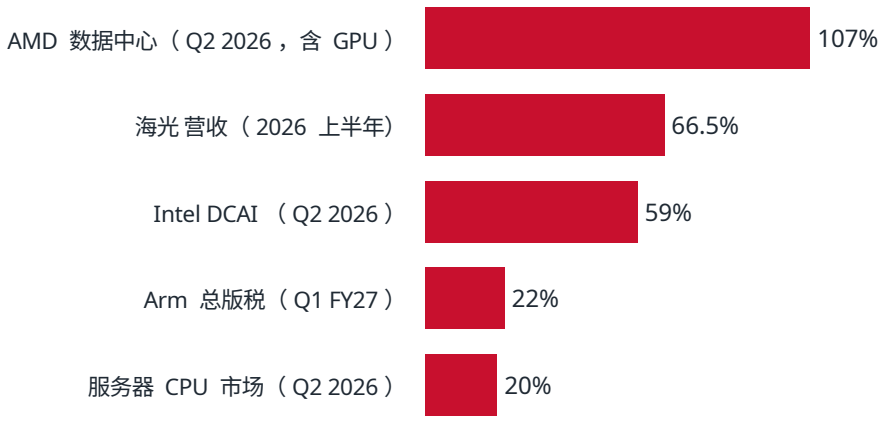
- 口径不一：CPU 颗数还是核数，整机群还是单台，分配的 vCPU 还是实际在用的核。
- 全部来自利益相关方，没有独立统计。
- 实测：每个沙箱平均约 0.1 核，Vera 机柜按约 1 核一个环境规划，差约 12 倍（第 44 页）。
- 柜内的 1:2 是为了内存；Agent 的增量在柜外 CPU 机柜。

方向一致：CPU 需求在涨；幅度没有一手数据。第五部分后面的负载数据，就是用来检验这些说法的。

数据来源与口径：Intel 与 AMD 的说法出自财报会议，经 TrendForce（2026-04-24、2026-05-06）、The Register、TIKR 转述；NVIDIA 出自 Q2 FY27 财报会议，经 The Motley Fool、Crypto Briefing 转述，“200 亿美元”是否只含独立 CPU 说法不一；Arm 与 TrendForce 的数字见 TrendForce 2026-04-14 的分析。图中数值按“GPU 颗数 ÷ CPU 颗数”换算。

市场信号（二）： CPU 厂商的业绩印证了数据中心 CPU 需求在涨——但增长里有多少来自涨价、多少来自 Agent，财报分不出来

最近一期的同比增速（%）



出货 +9%，售价 +48%

Intel 服务器 Q2 2026 同比（10-Q）

售价上涨主要来自高端产品占比提高，按需涨价是次要；需求超过供给，数据中心的供给紧张持续到 2027 年。 Arm 的数据中心版税同比翻倍以上（不披露绝对值）。

厂商与时期	数据中心相关数字	管理层的解释	能否拆出 Agent
Intel Q2 2026	DCAI 62.6 亿美元， +59%；服务器出货 +9%、平均售价 +48%； CFO 称少发货超过 10 亿美元（经报道）	AI 推动前所未有的算力需求；扩产 Xeon 6	增长主要来自价格与产品结构； Agent 未单列
AMD Q2 2026	数据中心 67.2 亿美元， +107%，占营收 58%；服务器 CPU 份额 34.5% （Mercury）	EPYC 与 Instinct 需求都强；下半年加速	CPU 与 GPU 合并披露
Arm Q1 FY27	营收 12.9 亿美元 +22%；版税 7.15 亿美元 +22%；数据中心版税翻倍以上	AI 基础设施转向 Arm； AWS 与 Meta 签约部署数千万个 Graviton5 核用于 Agent 负载	不披露数据中心版税绝对值
NVIDIA Q2 FY27	数据中心 890 亿美元； CPU 收入可见度约 200 亿美元（本财年）	Agent 使用增长带动数据中心 CPU 需求	是否只含独立 CPU，报道说法不一
海光 2026 上半年	营收 90.99 亿元 +66.5%；归母净利 17.98 亿元 +49.7%	大模型迭代、 AI Agent 规模化落地、国产化	CPU 与 DCU 合并披露
市场与供需	服务器 CPU 市场 Q2 同比约 +20%（Mercury）；交期从 1–2 周拉长到 8–12 周，部分 6 个月（2026 年初）	Intel、 AMD 在 1Q26 末对部分型号涨价	—

需求信号是真的：缺货、涨价、收入与份额同时上升；但没有一家把“因为 Agent”单独拆出来， Intel 的增长主要是价格和产品结构。对鲲鹏来说，能检验的是负载数据，而不是财报里的叙事。

数据来源与口径：一手： Intel Q2 2026 新闻稿与 10-Q （截至 2026-06-27）、 AMD Q2 2026 新闻稿（截至 06-27）、 Arm Q1 FYE27 新闻稿（截至 06-30）。转述： NVIDIA Q2 FY27 （截至 07-26） CPU 收入经媒体；海光半年报经新华财经（2026-08-13）； Intel 少发货、 Mercury 份额与市场增速经 HotHardware 等；交期经 Nikkei Asia、 Reuters；涨价经 TrendForce。各家的“数据中心”口径不同，增速不可直接比较。

从市场信号到负载分析：第五部分要回答四个问题—— CPU 用在哪一步、在不在数据中心、每个 Agent 要多少、哪些是税

1 CPU 用在 Agent 的哪一步?

市场的说法

编排、工具执行、子 Agent 协调都在 CPU 上（TrendForce 、 AMD ）

负载数据

工具执行、沙箱、编排、推理 host 都在 CPU 上；瓶颈在模型还是工具取决于负载和口径，工具时间占比 2–89%

详见第 45–46 页

2 这些 CPU 在不在数据中心?

市场的说法

默认都算进数据中心的 CPU 需求

负载数据

按 token ，本地执行约占一半，几乎不进数据中心；数据中心的沙箱 CPU 主要来自 RL rollout 和云端任务沙箱

详见第 42–43 页

3 每个 Agent 实际要多少 CPU 和内存?

市场的说法

Vera 机柜按每个环境约 1 核、 18 GB 规划

负载数据

实测平均约 0.1 核；内存是基线 185 MB 加 2–4 GB 的突发，内存先于 CPU 成为密度上限

详见第 44 、 51 页

4 哪些开销是税，鲲鹏能做什么?

市场的说法

“CPU 不够”，所以要更多 CPU

负载数据

新税落在单核速度、内存容量和执行环境创建上： tokenize 、推理调度、冷启动、状态驻留、向量检索

详见第 47–51 页与第六部分

市场说“ CPU 不够”，负载数据说“ CPU 平均用得很少，但关键路径上要快、内存先不够”——两者不矛盾，关键在超分配做多深和工具在哪里执行。

数据来源与口径：市场的说法见前两页；负载数据的口径见各对应页的页脚（ [56] TraceLab 、 [57] AgentSysBench 、 [R4] AgentCgroup 、 [R10] DSec 等）。

AI 时代负载的基本单位从“一次请求”变成“一个会话”；五类业务在 CPU 侧产生的税各不相同

Chatbot 时代 vs Agent 时代

维度	Chatbot 时代	Agent 时代
工作单位	单次请求，秒级	会话，秒到数小时；生产编码会话中位只有 20% 的时间在执行 [57]
GPU 负载	prefill 与 decode 并重	超长前缀加缓存读、输出很短：每次调用 6.8 万输入对 247 输出 [R2]
状态	基本无状态	KV 缓存、沙箱文件系统、向量库都要保留
CPU 角色	头节点，给 GPU 喂数据	跑沙箱、工具、编排、 RL 环境
瓶颈	GPU 算力	随请求漂移： GPU 、 CPU 、内存、网络都可能 [57]

五类业务在 CPU 侧的特征与主要的税

业务场景	量化特征	主要的税
传统微服务	Meta 一次 Fetch 中位 498 个服务块 [03]； Google 95% 以上应用流量是 RPC [17]	网络栈与 RPC 、序列化、压缩、拷贝、分配（第一至四部分）
AI 化业务（搜索、推荐、客服 RAG ）	调用图尾部多出 embedding 、检索、 rerank 、模型调用，约多 3–8 个调用（推算）； RAG 检索占端到端 <1%–86% [52]	网络栈与 RPC 、序列化、向量检索
训练与后训练	仅加载数据就占训练节点最多 40% CPU [53]； RL 环境准备占每实例 rollout 19–29% [76]	拷贝与搬移、压缩、加密、冷启动
MaaS 推理	vLLM V0 的 CPU 调度接近迭代时间 50% [49]；CloudMatrix384 每 2 颗 NPU 配 1 颗鲲鹏 [44]	推理调度、 tokenize 、拷贝与搬移
Agent	编码 Agent 每请求 8.8 次模型调用 + 10.8 次工具调用，上下文约 12 万 token [56]	全部，尤其 tokenize 、冷启动、状态驻留

读法：训练、 MaaS 推理、 RAG 在本页带过（细节见 AI_Datacenter_Tax_Kunpeng.pptx 与整合文档）； Agent 把 CPU 放回端到端的关键路径，本部分后 11 页集中讲它。

传统微服务照旧在跑； AI 化业务在调用图尾部多出模型调用；训练和推理的 CPU 侧税集中在 host ； Agent 让会话、工具和状态成为新的主角。

数据来源与口径：左表综合 ~/AgenticAI 《 Agentic 时代的数据中心》、 [57] AgentSysBench （ 35,037 个生产编码会话）、 [R2] GitHub Copilot 生产 trace ；右表取整合文档表 1-2： [03] Meta 、 [17] Google 、 [52] RAGO 、 [53] Meta 数据加载、 [76] SWE-MiniSandbox 、 [49] vLLM 、 [44] CloudMatrix384 、 [56] TraceLab 。

Agent 不是一个程序：大脑在 GPU 集群，主循环和工具跟着产品形态走；工具在哪执行，决定 CPU 负载在不在数据中心

大脑：模型推理

决定下一步做什么

在哪里运行：几乎总在 AI 公司的 GPU 集群

主循环： harness

调用模型、解析工具调用、拼装上下文

在哪里运行：跟着产品走：用户电脑或云端 VM

手：沙箱与工具执行

跑代码、跑测试、操作浏览器

在哪里运行：用户电脑、云端 VM 、沙箱平台或 POD 内 CPU 机柜

外部世界

数据库、搜索、各种 API

在哪里运行：通用业务区或互联网

主循环 → 大脑：一次模型调用；主循环 → 手：一次工具调用；手 → 外部世界：工具访问业务系统

典型场景

场景	大脑	主循环	手（工具执行）
本地 Claude Code	AI 公司 GPU 集群	用户电脑	用户电脑（进程级沙箱）
Claude Code 云端会话	AI 公司 GPU 集群	服务商托管的隔离 VM	同一台 VM
Codex Cloud	AI 公司 GPU 集群	服务商基础设施	每个任务一个隔离容器
Manus	所调用模型厂商的 GPU	Manus 自己的后端	自托管的 E2B microVM
Meta Muse	Meta 的模型	每用户一台专属云端 VM	同一台 VM
前沿实验室 RL 训练	实验室自己的 GPU 集群	训练框架	POD 内 CPU 机柜或第三方沙箱云
网页问答里的搜索	GPU 集群	API 服务器	共享的搜索与抓取服务（无沙箱）

三类工具：只有第三类落到数据中心的沙箱 CPU 上

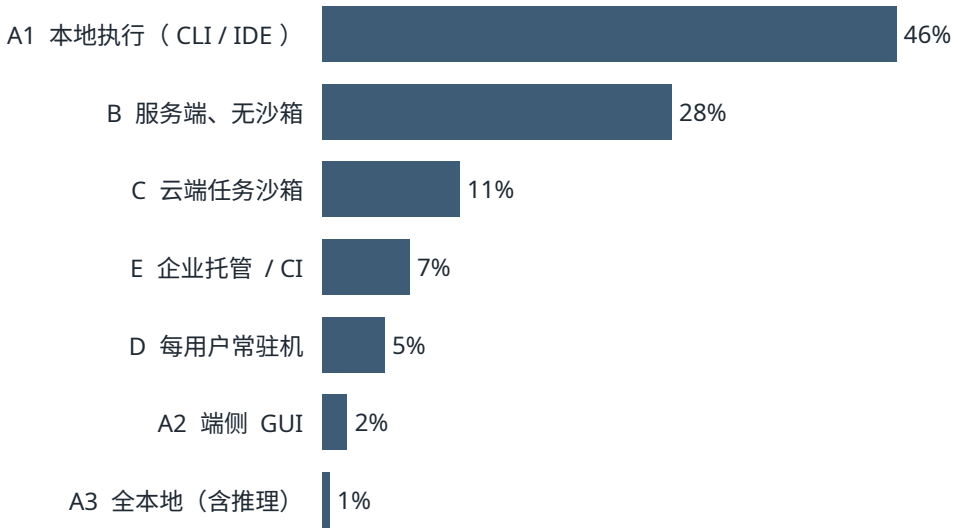
服务端工具（搜索、抓取）由 AI 公司的共享服务执行；客户端工具（本地 pytest ）由用户电脑执行；云端沙箱 Agent （ Codex Cloud 、 Manus 、 Muse ）由服务商分配的云端 VM 或 microVM 执行。

同一个编码 Agent ，本地运行时工具负载在用户笔记本上，换成云端会话、 RL rollout 或云开发机才进入数据中心——讨论 Agent 的 CPU 需求必须先说清是哪种形态。

数据来源与口径： ~/AgenticAI《 Agentic 时代的数据中心：负载、 CPU 部署与 agent 运行形态》（ 2026-10-08 ），依据各产品官方文档（ Claude Code 云端会话与沙箱、 Codex Cloud 、 E2B 的 Manus 案例、 Meta Muse ）；云端会话是否跑在 GPU 机房的 CPU 机柜上没有公开信息。

按 token 看，本地执行的 Agent 约占一半；按数据中心沙箱 CPU 看，排序几乎反过来—— RL rollout 和云端任务沙箱才是主要消耗者

Agent 类 token 的份额（中情景估算，%）



7.3 万亿 / 天

OpenRouter 上 Agent 每天的 token（7 日均值，8 月）

2 月 6 日首次超过人类；近七成是缓存读；9-21 前 20 个应用里本地执行工具的约占 82–86%（第三方平台，对全市场是上界）

每类的工具执行位置与数据中心沙箱 CPU

类别	工具在哪执行	生命周期	沙箱 CPU 排序	有生产数据
F 训练 / 评测 rollout	专用沙箱集群	随训练步突发创建	第 1	DSec [R10]
C 云端任务沙箱	每任务一个容器或 microVM	分钟到小时，空闲后暂停	第 2	Copilot [R2]（与本地混合）
D 每用户常驻机	每用户一台 VM	7×24 常驻，事件唤醒	第 3	Aries [R6]
E 企业托管 / CI	客户云中的 microVM 或 runner	会话级	第 4（记在客户云）	无
A1 / A2 / A3 本地	用户设备	交互会话	约为 0（用户机器）	TraceLab [56]（只有推理侧）
B 服务端、无沙箱	远程 API（搜索、抓取）	单次请求	约为 0（CPU 在共享服务上）	AgentSysBench 搜索问答 [57]

F 类不在服务态统计内，按 token 约为服务态的 5–30%；按 GPU 算力看比按 token 看大得多。

论文里“工具占会话时间 2–89%”是单个会话的时延口径，不能直接换算成数据中心的 CPU 需求：先看负载属于哪一类，再看这一类的沙箱能做到多高的密度。

数据来源与口径：份额来自团队表格《agent_ 分类与 token 占比估算》（2026-10-09）：按 12 个产品组的日均 token 推算，置信度低到中，不是统计数据；B 类边界取决于“带一两次搜索的对话”算不算 Agent。OpenRouter：Crypto Briefing 2026-08-10 原文为“7-day average”，与同期单日数据核对后判定为每天；应用榜见 openrouter.ai/apps [81]，偏开源模型与开发者。

CPU 部署分三层： GPU 柜内的 1:2 是为了内存，柜外的独立 CPU 机柜才是 Agent 带来的增量；而每个沙箱实测平均只用约 0.1 核

① GPU 柜内： 1:2 是为了内存，不是为了 Agent

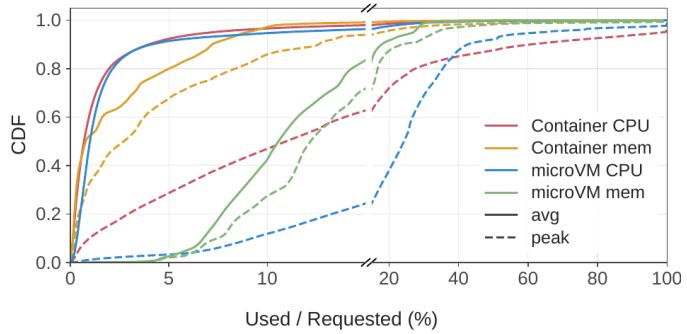
GB200 、 GB300 、 Vera Rubin NVL72 都是 36 颗 CPU 配 72 颗 GPU 。NVLink-C2C 900 GB/s 且内存一致， GPU 把 CPU 内存当扩展显存；每颗 GPU 摊到的 CPU 内存与 DGX H100 差不多（约 240–256 GB ）。

② 柜外：独立 CPU 机柜是 Agent 带来的增量

Vera Rubin POD 的 Vera CPU 机柜： 256 颗 Vera 、超过 22,500 核、 400 TB 内存，跑 Agent 沙箱与 RL 环境； CoreWeave 版 11,264 核； Arm AGI CPU 液冷柜超过 45,000 核。

③ 通用机群与用户本地

网关、数据库、搜索因工具调用而扩容；本地 Claude Code 的工具负载在用户笔记本上，换成云端会话才进数据中心。多数 Agent 沙箱今天仍跑在普通 x86 云主机或裸金属上。



DSec 生产：约 90% 的容器和 microVM 沙箱平均 CPU 不超过申请量的 5% （[R10] Fig. 5 ）

每个沙箱占多少

来源	每沙箱 CPU	每沙箱内存	性质
AgentCgroup [R4]	平均 0.08–0.13 核，突发超过 1 核	基线约 185 MB ，峰值 2–4 GB	单机实测，不限资源
DSec [R10]	约 0.08 核（ 3 万核 ÷ 38 万并发）	约 0.66 GB （ 250 TB ÷ 38 万）	生产，超分配后折算
Claude Code 云环境	4 vCPU	16 GB	单会话配额上限
NVIDIA Vera CPU 机柜	约 1 核	约 18 GB	厂商规格

CPU:GPU 走向 1:1 的说法（第 38 页）对超分配假设极敏感：按 DSec 的生产密度和按厂商规格算，每个环境的 CPU 差一个数量级。

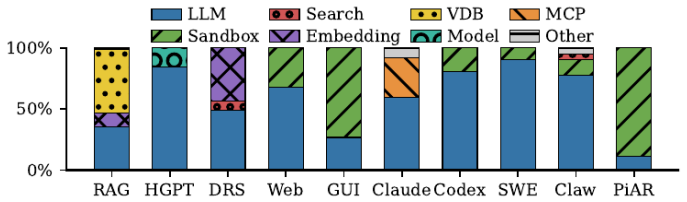
数据来源与口径：三层与 NVIDIA 、 CoreWeave 、 Arm 规格转引自 ~/AgenticAI《 Agentic 时代的数据中心》（厂商与分析机构数字，未逐项核实）； CPU:GPU 预测为 TrendForce （利益相关方）； [R10] DSec arXiv 2609.22978 （ DeepSeek ，一个生产单元约 160 节点）； [R4] AgentCgroup arXiv 2602.09345 （ SWE-rebench 144 个任务）； Claude Code 配额来自官方文档。

一次 Agent 请求的一生（和第 14 页同一画法）：每一步多跨了模型、执行环境、时间三条新边界；平均调 8.8 次模型、 10.8 次工具，每次把十几万 token 整段重发

① harness 拼上下文 → ② 调模型 tokenize · 调度 → ③ 执行工具 建沙箱 · 执行 → ④ 下一步 结果塞回上下文 → ⑤ 步与步之间 状态一直留着 → ⑥ 旁路 检索 · 日志

底色：紫 = AI 新增的三条边界（模型、执行环境、时间） 绿 = 存储边界 灰 = 进程内；②→④ 每请求循环约 9–11 轮

步骤	跨哪条边界	做了哪些操作	实测量级（口径）
① harness 拼上下文	进程内	维护 JSON 消息数组，把上一步结果追加进去	每步中位 11.9 万 token 是命中缓存的前缀，新增约 875 个（新内容约 0.8%， token 加权） [56]
② 调模型	模型（+ 服务）	整段上下文 HTTP+JSON 序列化 → 网关 → tokenize → 前缀块哈希 → 调度 → GPU → 每 token 一条流式小消息 → JSON 解析	每请求平均 8.8 次 [56]；前缀缓存命中 95.7% [56]； tokenize 每次调用核时 2.3–228 ms，取决于用哪个库 [68]
③ 执行工具	执行环境	MCP JSON-RPC 或本地命令 → 建沙箱或复用 → 执行（进程、文件、网络）	每请求平均 10.8 次 [56]； Codex 每条命令之外的开销均值 1.26 s（中位 0.16 s），比命令本身 0.66 s 还长 [56]
④ 进入下一步	进程内	把工具结果塞回上下文	上下文每步变长，所以②的单价每步上涨（第 49 页）
⑤ 步与步之间、等人时	时间	KV 缓存、沙箱内存、会话状态一直占着内存	生产编码会话中位只有 20% 的时间在执行 [57]；每个沙箱平均约 0.1 核（第 44 页）
⑥ 旁路：检索、日志、遥测	存储	向量扫描、日志	RAG 检索占端到端 <1%–86% [52]；编码 Agent 几乎不用



AgentSysBench 10 个应用的时间分解：多数以 LLM 为主， GUI 、 PiAR 以沙箱为主， RAG 以向量库为主（[57] Fig. 4）

Copilot 生产： 1,300 万会话、 7.61 亿次 LLM 调用； 87% 由 Agent 发起，模型与工具调用约 1:1；每次 6.8 万输入 token 对 247 输出 [R2]。

工具时长重尾： 中位 166 ms、均值 16.7 s [R2]；超过 1 分钟的调用只占 4.9%，却贡献 92% 的工具时间 [56]。

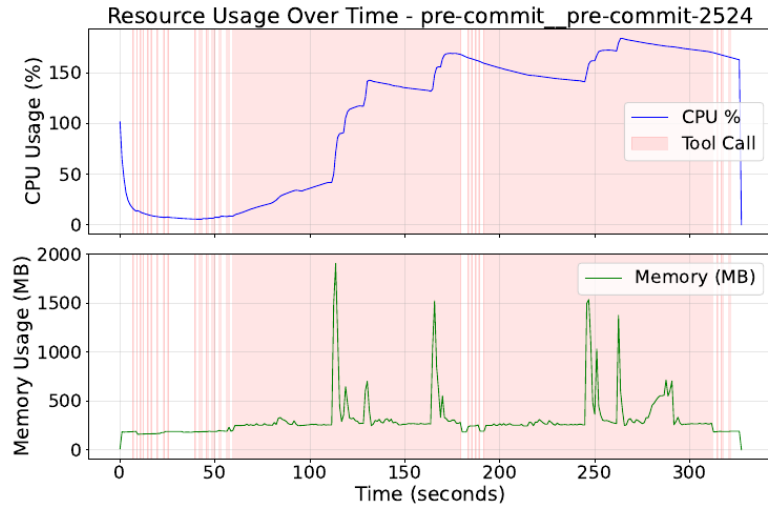
工具时间占比随口径从 2% 到 89%： TraceLab 请求内 59.8%； Copilot 只占会话墙钟 4.7%；检索、化学工具类可到 80% 以上 [56][R2][L3]。

和传统请求比有三处不同：调模型的单价随上下文变长而上涨；工具要先建执行环境；两步之间状态一直占着内存——正好对应三个专题。“瓶颈在模型还是工具”取决于负载类型和统计口径。

数据来源与口径： [56] TraceLab arXiv 2606.30560 （ Claude Code 、 Codex 真实使用， 43 位开发者 8 个月、 4,265 个会话）； [57] AgentSysBench arXiv 2608.15127 （ 10 个应用 + 阿里生产 trace ， 35,037 个会话）； [68] TokTier （ 117K token 上下文）； [R2] GitHub Copilot 生产 trace arXiv 2608.00101； [52] RAGO； [L3] CPU-Centric arXiv 2511.00739 。均值与中位已分开标注。

负载特征： Agent 对 CPU 的需求是平均低、突发高；先撞墙的是内存容量、突发核数、隔离和沙箱启动，不是峰值算力

需求	关键数字	证据
① 关键路径上的串行时延：单线程重新重要	工具步骤串行， Python 受 GIL 限制； GPU 越快 CPU 占比越高：换到 GH200 后 SWE-Agent 的 Bash / Python 从 25–38% 升到最高 65% [L3]	强
② 核数按突发供给，不按平均	编码 Agent 平均只用 7.6–22% 个核，突发持续 1–2 s [R4][R5]；工具容器 2→8 核，HotpotQA 时延降 13.0×、 web search 只降 1.04× [R5]	强
③ 每实例内存容量先于 CPU 成为密度瓶颈	峰值 2–4 GB、峰均比 15.4×； 128 GB 只能放 32–64 个实例，而 CPU 利用率不到 36% （“ memory, not CPU”） [R4]	强
④ 共享资源干扰与隔离	共享 128 核池比各绑 16 核慢 1.22× [57]； SMT 兄弟线程混部 +45.2%，开 core scheduling 后 +17.3% [R10]	中（ LLC、带宽缺归因）
⑤ 存储 I/O 与沙箱生命周期	容器与 Agent 初始化占任务 31–48%，镜像中位 3.5 GB [R4]；磁盘写 200 MB/s 已到上限 [R5]	中
⑥ 能耗： CPU 不可忽略	CPU 动态能耗最高 61% [L3]； SWE-bench 编码中 CPU 占系统功率 23.7%，只看 GPU 会漏掉 41–45% [R7]	中（依平台）
⑦ 编排与控制面：单项小、每轮都有	harness 编排最多占端到端 9% [R6]； Claude Code 的 Node.js 框架基线约 185 MB [R4]	中



AgentCgroup ： 一个任务的 CPU 与内存时间序列，内存随工具调用尖峰式跳动（[R4] Fig. 5 ）

形状是“长时间空闲 + 秒级尖峰”： 按平均配会被节流，按峰值配大部分时间闲置；可行的是深度超分配，再用调度隔离保护延迟敏感任务 [R10] 。

每核内存带宽、 PCIe 代际、机密计算的性能代价， 14 篇论文都没有直接测过——是假设，不是结论；微架构计数器（ IPC 、 MPKI 、前端停顿）的刻画也完全空白。

数据来源与口径：七类需求与证据强度取自 ~/AgenticAI 《 Agentic AI Workload 特征分析》 “ CPU 视角”一章（ 14 篇论文）；关键数字已对照原文： [R4] AgentCgroup 、 [R10] DSec 、 [L3] CPU-Centric 、 [R7] arXiv 2609.29707 、 [57] AgentSysBench ； [R5] arXiv 2609.19947 、 [R6] Aries arXiv 2607.29069 转引自该报告。

矩阵多了三列（一）：公式不变，只是多了三条边界；七个老税没有消失，而是在新的列上长出了新格子

税 ≈ Σ 边界 [跨越次数 × 每次单价 × 形态系数]　——　传统的服务、存储、进程 / 内核三列照旧，再加三条边界：
模型边界：调用次数 ×（tokenize(L) + JSON(L) + 块哈希 (L) + 调度 ）　　执行环境边界：创建次数 × 冷启动 × 隔离形态系数
时间边界：容量 × 驻留时间（内存 × 时间）＋ 搬移字节 × 每字节成本　　检索（存储边界的新形态）：查询次数 × 扫描字节 ÷ 内存带宽
L = 整段上下文长度：只有模型边界的单价随会话变长而上涨，这是和传统 RPC 最大的不同

老的行	传统：在哪一列、多大	新的格子：落在哪条新边界	新格子的量级或特征
通信（网络栈与 RPC 库）	服务列专属： TCP 栈 1.7% + RPC 库 1.1% （Google fleet） [02]	模型 / 服务边界：编排层多出一层更贵的调用—— HTTP+JSON 、每 token 一条的流式小消息、 MCP	生产链式 LLM 应用 30–50% 的延迟在模型引擎之外 [66]
序列化 / 解析	整行 9.6%，约八分之一在服务列，主体是拷贝 [25] [02]	模型与执行环境边界： JSON 以 Agent 消息和 MCP 的形式回来，主体变成“解析后建对象”	编码 Agent 每请求约 8.4 MB JSON 编解码（推算）；按字节 tokenize 比 JSON 贵 100–2,000 倍
拷贝与搬移	4–5%，多数 <512 B [01][20]	时间边界：要搬的从 KB 级载荷变成 GB 级 KV 缓存、快照、 checkpoint	8K 提示词的 KV 约 2.5 GB （Llama-3-70B，推算）；生产系统一律绕过 CPU 核 [44][45]
压缩	整行 2.9%，一半由文件格式库发起 [34]	时间与执行环境边界：快照、空闲内存（zswap）、 checkpoint；浮点张量直接压几乎无效	Intel IAA 压快照 2–4.5×、恢复快 25–55% [89]
内存分配	4.3%，不建议硬件化 [35]	模型边界（推理 host）：地址空间碎片化、 TLB 与页表遍历变贵	受干扰时页表遍历占末级缓存停顿 85% [58]
哈希	1–2% （2015） [01]	模型边界：前缀缓存的块哈希与内容寻址	每请求约 4 ms 核时，随上下文长度增长（推算）
加密	服务列 <0.2% （连接复用） [02]	执行环境边界：机密虚拟机把代价从 CPU 周期转到内存容量和 VM 退出	跨 VM 去重失效，内存多占 9–147% （x86 实测） [97]

读法：老的行在新列上长出新格子，所以吃整行的贴核方案在 AI 时代覆盖面变大—— KAE 压缩多了快照与 zswap 两格（第 58 页）；网卡 / UB 在新列里只找到 KV 搬移这一格（第 60 页）。

数据来源与口径：扩展式与表格取自整合文档 INSIGHT_DATACENTER_TAX_AI_ERA.md 1.1 、 1.3 节，按 v3 的矩阵重排；“推算”为该文档的显式估算； [66] Parrot OSDI'24； [44] CloudMatrix384； [45] Mooncake FAST'25； [58] Blink； [89][97] 闭门会文档语料（Intel IAA 快照、 x86 机密虚拟机实测）。

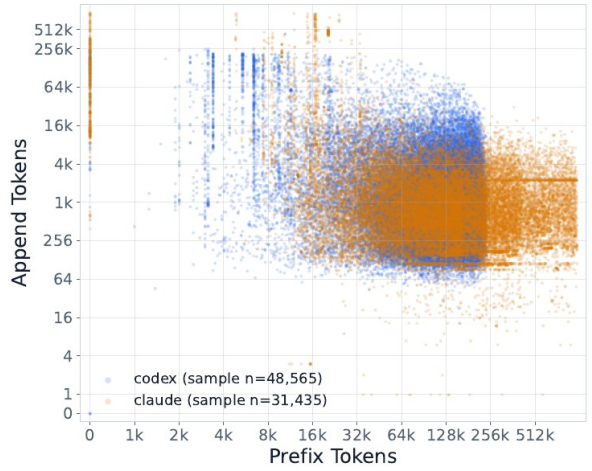
矩阵多了三列（二）：新列带来的专属行，即五个新税—— tokenize 、推理调度属于模型列，冷启动属于执行环境列，状态驻留属于时间列，向量检索是存储列的新形态

新税	是什么	业务特征	负载特征	量级（口径）	证据
tokenize （模型列）	每次调用模型前，把整段上下文重新切成 token	编码 Agent 每请求 8.8 次调用、每次约 12 万 token ，新内容约 0.8% [56]	单核串行：正则预分词加哈希随机查表	每次调用的核时： HuggingFace 228 ms 、Rust 13.4 ms 、增量 2.3 ms [68]	强；差别取决于用哪个库（ 17× ）
推理调度（模型列）	每个 decode step 的组批、launch 、同步、采样、 detokenize	MaaS 推理每一步都做； CPU 欠配普遍 [69]	单线程、 Python 与 C++ 混合，对单核、TLB 和共置干扰敏感	vLLM V0 接近迭代时间 50% [49] ； CPU 配少时首 token 慢 1.47–7.11× [69]	强；正被“绕过 CPU”削减
冷启动（执行环境列）	从零建出进程、解释器、容器、microVM 的固定开销	编码任务初始化占 31–48% [R4] ； RL 环境准备占 rollout 19–29% [76] ；本地 Agent 接近零	建页表、清零、动态链接、解释器初始化，冷的分支预测和 TLB	Codex 每条命令之外的开销均值 1.26 s （中位 0.16 s ），比命令本身 0.66 s 还长 [56]	中；云端与 RL 有实测，fleet 占比未知
状态驻留（时间列）	会话活着但空闲时， KV 、沙箱内存和会话状态一直占着内存	会话中位 20% 时间在执行；沙箱工作集中位 0.8 GB 、峰值 28 GB [57]	单位是 GB × 时间，不是 CPU 周期；内存峰均比 15.4× [R4]	192 核、 1.5 TB ：按内存峰值只能放约 510 个沙箱，按 CPU 约 1,340 个（推算）	强；生产会话实测
向量检索（存储列的新形态）	在百万到十亿个向量里找最近邻	RAG 检索占端到端 <1%–86% [52] ；编码 Agent 几乎不用	内存带宽型： PQ 扫描每核 18 GB/s ，用到约 80% 带宽 [52]	随场景从 <1% 到 86% [52]	强；带宽型结论稳

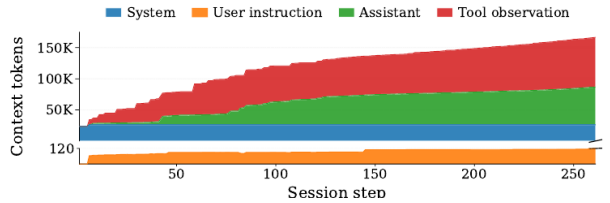
读法：老的行（第 47 页）可以用贴核的加速器吃整行；新列的专属行大多不适合做专用加速器，要靠通用核的单核速度、内存系统和软件（第六部分）。

数据来源与口径：五个新税的定义与数字取自整合文档表 1-1 第二部分： [68] TokTier arXiv 2607.29678 （表 8 ， 117K token 上下文，核时）； [49] NanoFlow / WukLab ； [69] arXiv 2603.22774 ； [R4] AgentCgroup ； [76] SWE-MiniSandbox （表 2 、表 4 ）； [57] AgentSysBench ； [52] RAGO ISCA'25 ；密度算式见第 51 页。

专题一：前缀缓存让 GPU 只算新增内容， CPU 这边却每次都把整段上下文重新 JSON 、 tokenize 、哈希一遍



TraceLab ： 每步的前缀 token （横轴）远大于追加 token （纵轴）（[56] Fig. 1 ）



一个生产会话的上下文随步数增长，工具输出占大头（[57] Fig. 21 ）

每次调用（约 12 万 token 、 480 KB ）的 CPU 账

环节	今天（常见实现）	做成增量以后	口径
tokenize	228 ms （ HuggingFace ）； 13.4 ms （ Rust 实现）	2.3 ms	核时， [68] 表 8
JSON 编解码	约 10 ms （ Python 标准库；每请求约 8.4 MB ）	约 0.2 ms （ simdjson 类）	推算
块哈希	约 4 ms （ 16 token 一块，约 7,500 块）	只算新增的块	推算
合计	约 242 ms	约 3 ms	推算

121–1,210 个

今天：每 1,000 张加速卡需要的前端核

每卡每秒 0.5–5 个请求（推算）

1.5–15 个

三项都做成增量以后

块哈希没做成增量时是 3–33 个（推算）

注： “前缀命中率 0.99 时 tokenize 占首 token 延迟 64%” 只见于 TokTier 的摘要和引言，正文没有给出扫描设置，引用时要注明 [68] 。

三项的根因相同：成本与整段上下文长度成正比，而不是与新增内容成正比。软件方向统一——做成增量；最彻底的是服务端保存会话、客户端只发新增（ OpenAI Responses API 已这样做）。

数据来源与口径： [56] TraceLab ； [57] AgentSysBench ； [68] TokTier arXiv 2607.29678 v3 （表 8 为 117K token 上下文的核时）； JSON 、块哈希与前端核数为整合文档 4.3 节的推算（每卡每秒请求数为假设）。

专题二：同样的工具负载，隔离方式不同，创建和系统调用的代价可差两个数量级——这是“部署形态系数”在 Agent 时代的对应物

隔离形态	创建一个环境	运行时的系统调用与 I/O	内存	出处
进程级限制（bubblewrap、seccomp、Seatbelt）	接近零	约 1×	无额外开销	[74]
容器（runc）	几十 ms（Daytona 27–90 ms）	比裸机慢约 32%	共享宿主内核	[64][81]
gVisor	几十到上百 ms	简单系统调用慢 2.2× 以上；外部 tmpfs 上开关文件慢 216×	用户态内核的额外占用	[64]（2019 微基准）
裸金属 microVM（Firecracker、CubeSandbox）	125 ms 内到应用代码；CubeSandbox <60 ms（厂商自报）	真内核，接近原生	每台额外 <5 MB	[51][106]
云主机里嵌套（PVM）	与单层接近	每次系统调用多约 210 ns；硬件嵌套高并发下会崩	影子页表开销	[92][93]
机密虚拟机	启动慢 102–394%	触发 VM 退出的指令贵 240–472%	跨 VM 去重失效，多占 9–147%	[97][109]（x86）

对照第二部分：RPC 时代的形态系数放大每次 RPC 的 CPU（library 1.36× → sidecar 2.7–3.7×）；Agent 时代的隔离形态系数放大每次创建和每次系统调用。

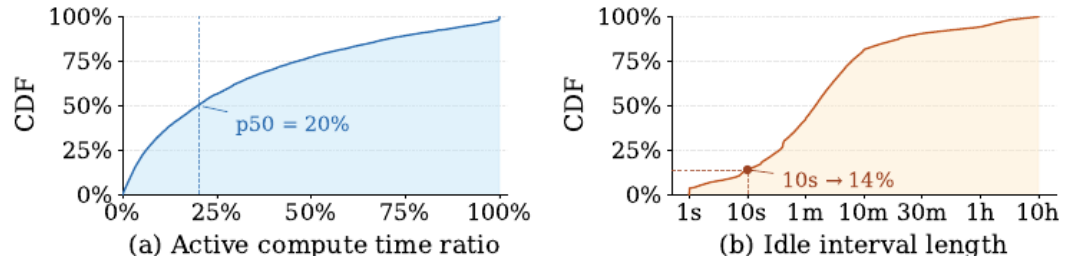
冷启动的实测

- 编码任务：容器与 Agent 初始化占任务生命周期 31–48%；镜像中位 3.5 GB，是典型微服务镜像的 7 倍、serverless 的 70 倍 [R4]。
- RL 训练：容器方案的环境准备占每实例 rollout 19–29%（64–256 个并行环境），进程级隔离降到 8–9% [76]。
- Codex：每条命令之外的开销平均 1.26 s（中位 0.16 s），比命令本身的 0.66 s 还长 [56]。
- DSec 生产：每秒创建 5,000 个以上；镜像运行时只读 4.2–13.3%；8,192 个容器突发创建时，完整拉取比按需加载慢 1.71× [R10]。
- 本地 Agent 用进程级沙箱，这一项接近零。

选哪种隔离由信任边界决定，同样不是由性能决定：云端要跑任意代码，所以冷启动税在云端会长期存在；能做的是让创建、快照和恢复更便宜。

数据来源与口径：隔离形态表取自整合文档表 1-4（[74] 进程级沙箱；[64] gVisor 与 REAP，2019 微基准；[51] Firecracker NSDI'20；[106] CubeSandbox 厂商自报；[92][93] PVM 与 ParaCell；[97][109] x86 TDX、SEV-SNP）；[R4] AgentCgroup；[76] SWE-MiniSandbox；[56] TraceLab；[R10] DSec。

专题三：会话中位只有 20% 的时间在执行，其余时间 KV 缓存和沙箱内存一直占着——内存容量比 CPU 先成为沙箱密度的上限



35,037 个生产编码会话：活跃计算时间占生命周期的中位 20%；空闲间隔从秒到小时（[57] Fig. 20）

59.4%

会话至少遇到一次缓存驱逐

重算约占总费用 31.5%（5 分钟 TTL）[57]

243 / 172 s

Copilot 跨回合空闲中位：容器 / KV

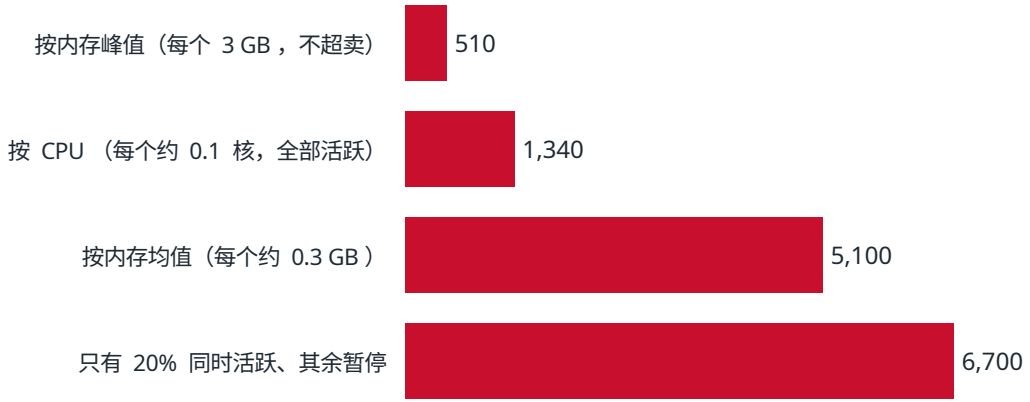
回合内只有 5.8 s，太短，不值得挂起 [R2]

0.8 / 28 GB

沙箱工作集：中位 / 单会话峰值

AgentSysBench 生产与受控实验 [57]

密度算式：192 核、1.5 TB 的一台机器能放多少沙箱（推算）



- 不超卖时，内存是第一个上限（约 510 个）；超卖做到约 2.6 倍，CPU 才成为上限；再往上，只有把空闲沙箱暂停、压缩或挪走才能继续增加。
- DSec 生产：共享宿主页缓存让峰值内存降 40.2%，回收空闲页让按时间积分的内存降 21.2% [R10]。
- KAE 压缩（zswap、快照）和 UB 远端内存决定超卖能推多远（第 58、59 页）。

状态驻留税的单位是“GB × 时间”，不是 CPU 周期；按 fleet 周期做的剖析量不出它，要用“每兆瓦能跑多少沙箱”这样的综合指标。

数据来源与口径：[57] AgentSysBench §7.1 与 Fig. 6、Fig. 20（35,037 个会话、24 小时）；[R2] Copilot 空闲时间表；[R10] DSec；密度算式为整合文档 3.4 节推算，资源画像只来自一篇论文（[R4] AgentCgroup：单机、单框架、不限资源）。

第五部分小结：同一张表多了三列和一种货币—— Agent 负载的六条洞察，以及哪些传统结论要改口

1 工作单位是会话，大部分时间在等人

生产编码会话中位只有 20% 的时间在执行；人的思考占会话时间 92.3%。

2 每次调用都整段重处理（模型列的单价随 L 上涨）

tokenize、JSON、块哈希的 CPU 成本与上下文长度成正比，新内容不到 1%。

3 瓶颈在模型还是工具，取决于负载和口径

工具时间占比 2–89%；不能把单会话时延直接换算成数据中心 CPU。

4 CPU 需求平均低、突发高，先撞内存容量

每个沙箱平均约 0.1 核，内存峰均比 15.4×。

5 数据中心的沙箱 CPU 来自 rollout 和云端沙箱

按 token，本地执行约占一半，但几乎不进数据中心。

6 新税多数不适合做专用加速器

靠单核速度、内存系统和软件增量化。

哪些传统结论要改口

传统结论（第一、二部分）	AI 时代的修正
传统微服务的内存带宽有富余（利用率中位 10%）	仍成立，但只适用于传统微服务；Agent 先撞内存容量，向量检索是带宽型，每核带宽是否成瓶颈待测
fleet 里省 CPU 比省 µs 重要	首 token 关键路径上的 tokenize 和调度是串行单核，单核速度和时延重新排到前面
只有跨应用的税值得硬件化	仍成立；但新税多数不适合专用加速器，要靠通用核、内存系统和软件
部署形态系数：library → sidecar → serverless	对应物是隔离形态系数：进程级 → 容器 → gVisor → microVM → 机密虚拟机
SMT 有吞吐空间	生产测得兄弟线程混部的尾延迟代价 +45.2%（core scheduling 后 +17.3%）
用 fleet 周期剖析衡量税	量不出“加速卡空转”和“内存被占着”，需要“每千卡前端核数”“每兆瓦沙箱数”两个综合指标
骨架：服务、存储、进程 / 内核三列	同一张表多了模型、执行环境、时间三列和“内存 × 时间”这种货币；只有模型列的单价随上下文长度上涨

一句话：Agent 把 CPU 放回关键路径，但它要的不是更多的峰值算力，而是更快的单核、更大更便宜的内存、更便宜的执行环境——第六部分按这三点规划。

数据来源与口径：各条的数字与口径见第 45–51 页页脚；修正一栏对应第 11、20 页的传统结论。

六

规划：按矩阵把发力点落到硅上

落地规则：行 → 核内指令与 KAE；列 → UB 与 DPA；次数与系数 → 软件与协作团队；内存 × 时间 → 内存系统；流水线 → 核总图（税 × 硅）→ 核 → 核内指令与系统库 → KAE 与 DPA → 内存系统 → UB → 协作项 → 优先级 → 950 待测清单（矩阵的空格）

方法：按矩阵把发力点落到硅上——行交给核内指令与 KAE，列交给 UB 与 DPA，次数和系数交给软件与协作团队；每条分 950 软件与 960 硬件

方法与边界

落地规则：行（通用操作）→ 核内指令、系统库与 KAE；列（服务边界的通信、 KV 搬移）→ UB 与 DPA；次数和系数 → 950 软件与协作团队；内存 × 时间 → 内存系统；流水线 → 核。同一个硬件常服务好几行（如 KAE 服务压缩、加密和状态驻留），按硅写一次。

两半：“950 现在做”是软件、配置、生态和测量；“960 硬件”是软件做完以后仍然剩下、需要核、 SoC 、加速器或互连解决的部分。

归属：只有标“基础加速”的是本团队的工作；海思、欧拉、 UB 、沙箱、推理框架等团队的事写成协作项，本团队提供分析和 950 上的测量。

950： 96 核 192 线程与 192 核 384 线程两款； 256 位 SVE2 （带 BitPerm 、 SVE-AES 、 SHA3 、 SM4 、 I8MM 、 BF16 ），没有 SVE2.1 、 MOPS 、 CSSC ；每核 L2 1 MB ，两个 SMT 线程共用；支持 MPAM 。

960： 2028 年， 96 核高单核款与 256 核以上高密度款。 920 即将停止服务，只作历史经验引用。

缺的数据：内部微架构参数、 KAE 小粒度时延、 UB 小消息 Mops 都没有，汇总在第 63 页的待测清单。

与市场信号的关系： CPU:GPU 比例的说法与厂商业绩见第 38–39 页；这里的规划依据第五部分的负载数据，不依赖这些叙事。

对照系： Intel 片上加速器、 NVIDIA 的 Agent 平台 vs 鲲鹏

参照	做什么	鲲鹏对应物	状态
Intel DSA	拷贝 / 搬移、填充、 CRC 、比较	片上 DPA （数据搬移加速器）〔内部口径〕	对应；软件接口与小段提交延迟待测
Intel IAA	压缩 / 解压 + 内存分析（扫描、过滤）	KAE ZIP 覆盖压缩；内存分析无对应	压缩有；分析空白
Intel QAT	加密 + 压缩	KAE SEC / HPRE / ZIP （片上共享，免许可）	已覆盖
Intel DLB	核间硬件队列与负载均衡	NI 侧有 UB Jetty Group ；核侧无对应	核侧空白（ ARM 阵营共有）
序列化加速器	protobuf 编解码	无	没有任何厂商量产；文献原型 ProtoAcc 、 Cerebros
NVIDIA Vera CPU 机柜 + BlueField-4	Agent 沙箱与 RL 环境的独立 CPU 机柜； DPU 管 KV 存放	鲲鹏 + UB 内存池 + KAE	Agent 时代的直接参照

一个坐标系：压缩和加密（ KAE ）已有，搬移（ DPA ）的软件接口待打通；序列化和核侧队列是空白； Agent 时代的参照是 NVIDIA 的独立 CPU 机柜。

数据来源与口径： Intel 加速器： Sapphire Rapids 公开资料；鲲鹏： KAE 仓库（ gitcode.com/boostkit/KAE ）、 openEuler UADK 文档、华为 HC2025 〔公开〕； DPA 为内部口径（ 2026-09-29 ）； 950 指令集见笔记 [83] ； NVIDIA 规格转引自 ~/AgenticAI 文档。

6.2 总图 · 税 × 硅（● 主要落点，○ 次要）

总图：按矩阵把税分四组——通用操作的红格落在核内指令与 KAE （吃整行），通信落在 UB， AI 新税落在单核、内存和执行环境创建

税 / 特征	核：前端与单核	核内指令与系统库	KAE	DPA	内存系统	UB	协作：内核与虚拟化	量级（口径见附录 B）
通用操作（矩阵的行，每条边界都会用）：贴核的方案吃整行								
序列化 / 解析		● SVE2 文本库				○ SG 落 UB		整行 9.6%； Agent JSON
压缩			● 4 KB 粒度		○ zswap			整行 2.9%；快照、空闲内存
拷贝与搬移		○ MOPS		● 小段 + SG		● KV 直传	○ 快照脏页	4–5%； GB 级 KV
内存分配	○ TLB	● 分配器 + 大页			○ 大页			4.3%；推理 host
哈希		● SHA 、增量						1–2%；每请求约 4 ms
加密			● 网关、快照				○ 机密 VM	<0.2%；机密化 +9–147% 内存
边界专属操作：服务列的通信，以及模型、执行环境、存储列的新税								
通信（服务列）	○ 唤醒切换		○ 网关 TLS	○ 拷贝		● RPC 感知	○ 内核网络栈	2.8%； RPC 税见第 21 页
tokenize （模型列）	● 单核、 L1/L2	● 增量库			○ MPAM 隔离			每次 2.3–228 ms 核时
推理调度（模型列）	● 单核				○ TLB 、 MPAM	○ CPU→NPU		V0 接近迭代 50%
冷启动（执行环境列）	● 前端、睿频	○ 用户态 spawn					● 页表、清零	初始化 31–48%
向量检索（存储列）					● 带宽	○ 索引溢出		<1%–86%
第四种货币：内存 × 时间（时间列）								
状态驻留			● zswap 、快照	○ 脏页搬移	● 容量、超卖	○ 远端慢层	○ HAFT 、 TLBID	按内存约 510 个 / 台
第三种切法：流水线状态								
前端（微架构特征）	● 取指、 BTB							slot 15–30%

读法：第一组的红格集中在核内指令与 KAE—— 吃整行；第二组里服务列的通信落在 UB， AI 新税落在单核和协作项；第三组落在内存系统。这就是第 27 页放置规则在鲲鹏上的样子。

核：前端取指和突发单核速度是传统微服务和 Agent 工具负载共同的第一诉求——软件改不到，只能靠 960

方向	传统依据	AI / Agent 依据	950 现在做	960 硬件	归属与优先级
前端取指（ L3→L1-I 带宽、 BTB 、 ITLB ）	前端停顿占 slot 15–30% [01] ； 生产 L1-I MPKI 7–56 [05]	编译、测试、短命进程、解释器每次都冷启动；工具执行在关键路径上 [L3]	用编译、 pytest 、 git 、 bash -c true 和 DCPerf 做 top-down ： 取指、 BTB 、 ITLB 未命中	更高的 L3→L1-I 取指带宽、更大的 BTB 和 ITLB ； 按真实负载而非 SPEC 调优	海思决定，本团队提供负载与测量； P0
突发单核与 SKU 分档	Uber ： 主频 +9% → P50 −15% [07]	tokenize 、推理调度是首 token 关键路径上的串行单核活 [68][49] ； 工具步骤受 GIL 限制 [L3]	测固定频率与允许睿频两种情况； tokenizer 独立进程、独立核	按核睿频、快速 DVFS ； 96 核高单核款给推理 host 与云开发机， 256 核以上款给 rollout 与沙箱平台	海思； P1
SMT 与 MPAM	叶子函数 IPC 不到峰值一半， SMT 有吞吐空间 [20]	兄弟线程混部 +45.2% ， core scheduling 后 +17.3% [R10] ； 超卖做好后 SMT 的吞吐增益直接变成沙箱数	tokenizer 与推理引擎共置的 MPAM 实验； 高优先级沙箱开关 SMT 的 P99	SMT 两线程之间能分 L2 ； CMAX 与监控计数齐全； MPAMv2	tokenizer 隔离本团队提， MPAMv2 协作； P1
一致性域与核间	跨 LLC 域 2.07× [35] ； 384 核内核锁争用 54% [05]	TLBI 广播： 24 台 VM 时性能掉到 0.62 （第 61 页）	同 die 与跨 die 的 RPC 往返；核间 ping-pong 热力图	可划分一致性域、向软件暴露 cache 域拓扑	海思、欧拉； P2
核侧队列与唤醒	切换 5K 周期、 >50 µs 唤醒伤尾延迟 [36][02] ； nanoPU −5× 、 µManycore −10.4×	—	测 WFE→ 唤醒延迟与 C-state 退出延迟	消息驱动唤醒（ WFET 类）、硬件线程队列（ DLB 类）	海思； P2 （只对 µs 级服务有效）

920 的历史经验〔公开〕： Chips and Cheese 测得 920 的 L3 取指带宽差（接近从 DRAM 取指）、 64 项单周期 BTB 、 L2 TLB 11 周期—— 950 要先测这几项是否补上。

前端和单核的共同点是传统微服务与 Agent 工具负载都要、而且软件改不到——这才是它们排在 960 核诉求第一位的原因，而不是“量级最大”。

数据来源与口径：文献见括号； 950 、 960 规格：华为 HC2025 与笔记 [83] ； 920 微架构： Chips and Cheese 〔公开〕；优先级与归属沿用整合文档表 4-1 。

核内指令与系统库： tokenizer 是本团队能碰到的最大一块 CPU 开销；文本处理指令 950 已够用，缺的是跨寄存器搬运时延和 MOPS

方向	传统依据	AI / Agent 依据	950 现在做	960 硬件	归属与优先级
增量 tokenizer 库	—	每次调用 228 ms 核时（HuggingFace），增量方法 2.3 ms [68]	线性 BPE、紧凑表、SVE2 预分词、会话状态；与参考实现逐位一致；先做 HuggingFace 的 drop-in 替换	低的 L1 / L2 命中时延；不做 tokenizer 加速器	基础加速； P0
进程启动的用户态路径	serverless 冷启动 1.2 s → 4 s （加策略） [43]	Codex 命令外开销均值 1.26 s [56]；每请求 5–7 个 shell 类进程	libuv 改用 posix_spawn、默认紧凑重定位、预 fork 进程池	前端取指与睿频（上页）	基础加速； P1 （RL 客户 P0）
推理 host 的分配器	分配 4.3%，硬件卸载只 +1.86% [35] [20]	页表遍历占末级缓存停顿 85% [58]； Arm 上换 mimalloc 快 2.3× [74]	Arm 调优的 mimalloc / jemalloc 加大页后端；框架缓冲区大页接口	更大的 L2 TLB、页表遍历缓存（协作）	基础加速； P1
JSON 与文本库	protobuf 9.6% [25]	simdjson 在 Neoverse 上每核 5–7 GB/s，慢在 Python 路径 [81]	256 位 SVE2 用进 simdjson、simdutf、base64；快速 JSON 库的 Python / Node 绑定	不做 JSON 加速器	基础加速； P2
块哈希	哈希 1–2% [01]	每请求约 4 ms， tokenize 增量化后比它还大（推算）	SHA-256 指令、 SVE 版 xxh3 / blake3；与 tokenizer 一起增量化	无（950 已有 SHA、 SHA3 指令）	推理框架、基础加速； P2
近核序列化微引擎	9.6%，可加速 3.45%；贴核 0.41 mm^2 [25]	JSON 已经很快，不适用	先用 zBuffer 类软件零拷贝落 UB SG；在 950 上跑 HyperProtoBench 定量	ProtoAcc 类近核引擎，只为 protobuf 立项	海思； P2 （理由见第 62 页）
指令集基线与跨寄存器时延	memcpy 4–5% [01]	950 没有 MOPS、 SVE2.1、 CSSC； UMOV 在 Neoverse V2 上最多 10 周期 [83]	缺的指令都有替代写法：这是软件缺口，不是硬件诉求	基线定到 Armv9.6 （MOPS 为 P1）；向量 / 谓词到通用寄存器的搬运做到约 2 周期	本团队提； P1

“不做”的清单同样重要： tokenizer、 JSON、正则都不做专用加速器；文本处理指令 950 已够用，硬件上真正的问题是跨寄存器搬运时延。

数据来源与口径： [68] TokTier 表 8； [56] TraceLab； [58] Blink； [74] vLLM-on-Arm 等 web 来源； [81] Agent 市场与生态 web 来源（simdjson 数据）； [83] 950 指令集与 Neoverse 时延； [25] ProtoAcc；归属与优先级沿用整合文档表 4-3。

KAE 与 DPA：传统的“KB 级同步压缩”和 AI 的“4 KB 页压缩”要的是同一件事——小粒度下的固定开销足够低

KAE 现状〔公开〕： ZIP （zlib 、 gzip 、 zstd 、 lz4 、 snappy ； 大块吞吐压缩 7 GB/s 、解压 8 GB/s ）、 SEC （AES 、 SM4 、 SM3 ）、 HPRE （RSA 、 SM2 ）； 接入 OpenSSL 、 zlib 兼容 API 、 UADK （SVA ， 队列式提交）； 片上共享、免许可。位置比 PCIe 卡近（PCIe 只有近核的约 1/5.6 [34] ），比贴核远，接近 CDPU 的 chiplet 情形（9.5× 对 10× ）。

方向	传统依据	AI / Agent 依据	950 现在做	960 硬件	归属与优先级
RPC 与存储路径的压缩	约 3% fleet ； 关键服务 10–50% ； 95% 字节用弱算法，加速后可换回容量 [34]	checkpoint 、镜像、训练数据层	测 1–64 KB 盈亏粒度（目标 4 KB 同步往返 1–2 μs 内）； 接 gRPC / bRPC / HCOM 压缩点	4 KB 粒度的固定开销接近大块吞吐； 预置 Huffman 表	基础加速； RPC P2 、checkpoint P1
zswap 与沙箱快照压缩	—	不超卖时内存是密度第一上限（约 510 个 / 台）； Intel IAA 压快照 2–4.5× 、恢复快 25–55% [89]	KAE 做 zswap / zram 后端； 按 4 KB 页、静态 Huffman 表压快照	加速器侧地址翻译不缺页（缺页时解压慢约 2× [89] ）	基础加速； P0
链式执行	同步对异步差 1.3× ， 链式 ≈ 全异步 [41]	快照： ZIP → SEC → SM3 不回核	在快照集成库里串起三个引擎	引擎间链式描述符、共享缓冲	基础加速； P0 （随快照）
加密	连接复用后 <0.2% ； 贵在建连和网关 [02][21] ； 阿里云弃用 QAT [24]	训练数据层 TLS ； 机密虚拟机	KAE 接网关 TLS ， 提供单请求同步、不攒批模式； 国密作差异化	低并发、小批量的加密路径	基础加速； P2
DPA 小段搬移	memcpy 4–5% ， 多数 <512 B [01] ； DSA 卸载 − 55% 主机周期 [28]	快照的脏页收集、恢复、写时复制、页清零都要批量搬移 [105]	用户态低时延提交接口， 支持 SG 批量描述符； 测与 SVE memcpy 的交叉点	64 B–4 KB 固定开销低于 100 ns ； 与 KAE 共享描述符和地址空间	基础加速； P0
序列化（KAE 空白）	9.6% ； 调用粒度百字节， 约 84% 不在 RPC 路径 [25]	Agent 的 JSON 已经很快	不用 KAE 队列模型做； 软件走 zBuffer 类零拷贝落 UB SG （编解码约 1/23 [19] ）	见上页近核引擎	—

KAE 的四条 960 诉求（4 KB 固定开销与预置 Huffman 表、引擎间链式描述符、加速器侧翻译不缺页、低并发小批量加密）和 DPA 的两条，由本团队直接提——使用者就是本团队的软件栈。

数据来源与口径： KAE： gitcode.com/boostkit/KAE 、 openEuler UADK 〔公开〕； DPA： 内部口径； [34] CDPU ； [41] Google ISCA'23 ； [89] Intel IAA 快照（闭门会语料）； [105] Copier 等； [28] RpcNIC ； [19] zBuffer ； 归属见整合文档表 4-3 。

内存系统：传统负载的带宽有富余， Agent 沙箱先撞容量； 960 要在容量、带宽和功耗之间重新取舍

方向	传统依据	AI / Agent 依据	950 现在做	960 硬件	归属与优先级
容量与超卖	—	内存约占服务器成本 53% （二手）；按内存峰值每台约 510 个沙箱（推算） [R4]	KAE zswap ；冷热识别、异步迁移、快速回收；密度基准测试	容量、带宽、功耗一起取舍（ DDR 通道、 LPDDR5X / SOCAMM2 、 UB 通道补带宽）	基础加速、欧拉、海思； P0 / P1
UB 内存池作慢层	带宽有富余但热页集中 [01][05][40]	空闲会话状态、 KV 溢出、向量索引溢出	UB 远端内存（ OBMM 上线为 NUMA 节点）作慢层的实测	远端内存按缓存行粒度访问；本地与远端请求公平调度	UB 团队、 KV 存储团队； P1
带宽	利用率中位 10% 、 95 分位 31% [01]	向量检索是带宽型（ PQ 扫描每核 18 GB/s ） [52] ；每核带宽是否成瓶颈未测	沙箱密度满载点测每核带宽利用率（是否越过约 50% 的拐点）	192 核款每核约 3.2 GB/s （推算），按实测再定	海思； P1
MPAM 隔离	混部：主机利用率 10→30% ， RT 劣化 20% [04]	tokenizer 与推理引擎共置；数千沙箱各自分份额	vLLM / MindIE 加满负荷 tokenizer 的隔离实验（只绑核、加缓存分区、加带宽分区）	SMT 线程间分 L2 ； MPAMv2 的 PARTID 虚拟化	tokenizer 隔离本团队提；海思； P1
TLB 与页表	分配 4.3% 不硬件化 [35]	推理 host 页表遍历占末级缓存停顿 85% [58]	大页后端；测 MindIE / vLLM 的 TLB 未命中率	更大的 L2 TLB 、页表遍历缓存、 BBML2/3	海思；协作
冷热识别	—	会话中位 80% 时间空闲 [57]	欧拉的冷热识别与回收	HAFT （与 SMMU 一起做）、内存侧冷热计数	海思、欧拉； P1

“带宽过配”和“每核带宽”不矛盾：传统负载今天带宽有富余；高密度沙箱在超卖做好、核被塞满之后，每核带宽才可能成为下一个瓶颈——需要 950 实测确认。

数据来源与口径：内存占服务器成本 53% 为二手（ 2026-07 ，笔记 [81] ）；密度与每核带宽为整合文档 3.4 、 4.1 节推算； [52] RAGO ； [58] Blink ； [57] AgentSysBench ； [R4] AgentCgroup ；归属见整合文档表 4-1 、 4-4 。

UB： 传输层已具备文献要求的大部分特性，缺口在 RPC 感知和硬件分发； AI 侧要先实测已有的 Mooncake 鲲鹏后端

文献需求	UB 对应机制〔本地〕	状态
消息语义、每消息极小状态（避免连接 cache 悬崖）[31][38]	RM 模式无连接，每条 WR 自带目标； TP Channel 按节点对共享	有
松弛排序与拥塞感知多路径（ Falcon vs RoCE 8× 的来源） [30]	ROI / ROT / ROL / UNO 可选； TPG ≤32 通道逐包 spraying	有
不依赖无损网络	RTP 端到端重传 + CC； CTP 依赖链路层无损（超节点内）	有
延迟型 CC 、硬件时间戳、接收侧占用回传 [30]	传输层 CC 、 VL 信用、 RNR； CQE 是否带时间戳待查	部分
按 RPC （非按连接）打 QoS [17]	SL / VL 存在； ubsocket 是每链路一个优先级	部分
按核实时负载的硬件分发 [11]	Jetty Group 按 RQ 深度分发；有界队列与按方法分发待测	有（ RQ 深度）
用户态权限、免注册 [26]	UMMU MAPT 可下放、 non-pin SVA； UMQ 整池注册一次	有
写触发的轻量通知、免 doorbell 提交 [12][14]	JFCE 完成事件； DWQE 用 st64b 直写 SQE	有

bRPC over UB 与 AI 数据路径

bRPC over UB 已落地〔本地〕： TCP 握手后切到 UMQ， IOBuf 挂 UMQ 注册内存池。四个评估点：默认 RC 改 RM，看数千对端下的延迟悬崖； ubsocket 最小块 8 KB 而中位 RPC <1.5 KB，要小消息 inline； zBuffer 字段直接成为 SGE；建连成本在短连接下放大。

AI 数据路径： Mooncake 主线已有 kunpeng_transport （URMA）和 ubshmem_transport，先实测而不是重做 [75]； xDeepServe 的 P/D 间 KV 直传已走 UB 内存语义（ XCCL） [60]； CloudMatrix384 的 EMS 用 UB 池化鲲鹏内存存 KV [44] 。

最需要的是数据： 每核 Mops 、 RTT 、每 Gbps 核数、连接数对延迟；对标 eRPC 2.3 µs 、 10 Mrps/ 核， Falcon 120 Mops， Homa/Linux 100 Gbps 需 18 核。

UB 的 960 方向： RPC 感知（按消息映射服务等级、完成事件带时间戳、可编程拥塞控制、 Jetty Group 有界队列）、 CPU 的 UB 通道数与内存带宽匹配、远端内存按缓存行访问——归 UB 团队与海思。

数据来源与口径： UB 机制： ~/ub_doc_and_code 、 ~/umdk 、 ~/ubs-comm 、 ~/brpc_ub 〔本地〕； UB 2.0： 华为 HC2025 〔公开〕； [75] Mooncake UB 后端现状； [60] xDeepServe； [44] CloudMatrix384 ； 文献需求来自 Falcon 、 Homa 、 eRPC / Snap 、 Aequitas 、 RPCValet 、 Cornflakes 、 Dagger 、 HydraRPC 。

协作项：高密度沙箱的系统开销归海思、欧拉和沙箱团队，本团队提供 950 上的测量

诉求	解决什么	证据	建议	归属
TLBID （TLBI Domains）	ARM 的 TLBI 默认广播全片，VM 多了性能下滑	华为 MAD24： 96 核上 24 台 4 核 VM 时 unixbench 掉到 0.62，按亲和限定广播后 0.95 [94]	960 支持 TLBID 列为 P0； 950 先用好 TLBIRANGE 与批量刷新	海思；软件归欧拉
HDBSS / HACDBS 、 HAFT	硬件记录并批量清除脏页、硬件访问标记	x86 上 PML 让检查点缩短最多 10.18% [105]	随快照与冷热识别一起要； HAFT 要 SMMU 同步支持	海思、欧拉
GICv5 与用户态中断	上千个睡着的 vCPU 唤醒更便宜； ARM 没有 UINTR 等价物	Intel 用户态中断接收 600–900 周期，信号约 2,800 周期 [99]	先问清客户用途；替代是 WFET 、 GICv5 直接注入、 UB 的 JFCE	海思
嵌套虚拟化与 ARM 版 PVM	在租来的 ARM 云主机里跑沙箱； CubeSandbox 的 PVM 只支持 x86 [106]	x86 上硬件嵌套高并发会崩， PVM 快两个数量级 [92]	欧拉评估 ARM 版 PVM； 950 若实现 NV2，在华为云开放嵌套是最短路径	欧拉、海思
轻量隔离（MTE 、 POE 、 CHERI）	能否用比 VM 更轻的机制达到 VM 级隔离	MTE 真实芯片最坏慢 6.64×； POE 只有 7 个可用域 [95][96]	保留 VM 边界，在内部用 POE 把切换做轻（ParaCell 思路） [93]	欧拉、海思
SMMU 规模	一台 2,000 个沙箱都要网络和存储， VF 只有几十到几百个	PASID 队列级隔离让虚拟设备数提高 2.96× [100]	请海思给 StreamID / SubstreamID 规模表；本团队先用 KAE 队列（SSID）测	海思；本团队测量
进程创建的内核路径	fork 页表复制、页清零、缺页	大进程 fork 比 vfork 慢最多约 300× [81]	本团队在 950 上测拆分，内核与沙箱团队合并提	欧拉、沙箱团队
机密虚拟机	沙箱对云厂商也保密	x86： 启动慢 102–394%，去重失效多占 9–147% [97]	在 950 上测 virtCCA / CCA 的四个数；同一机密 VM 里放多个沙箱	机密计算团队
安卓沙箱	手机 Agent 的 RL 训练和评测	每个安卓虚拟设备 5 GB 以上内存； ARM 可原生跑安卓 [111]	在 950 上测 Redroid 与 microVM 内安卓的密度，对比 x86 加翻译层	欧拉、沙箱团队、华为云云手机

这些都不是“税”，也不在本团队的软件栈里；本团队的角色是把 950 上的测量做出来，让海思、欧拉提诉求时有数据。

数据来源与口径：取自整合文档表 4-2 与闭门会文档 INSIGHT_KUNPENG_NEXTGEN_SANDBOX.md 第 3 节； [94] 华为 Linaro MAD24 TLBI 报告； [105] G3 快照集合； [99] UINTR ； [106] CubeSandbox ； [92][93] PVM 、 ParaCell ； [95][96] POE 、 MTE ； [100] HD-IOV ； [97] 机密虚拟机； [111] 安卓沙箱。

优先级：先测量建基线；本团队的 P0 是增量 tokenizer、KAE 的 4 KB 压缩（快照与 zswap）和 DPA 接口；960 的第一诉求是前端取指

950：本团队的软件（基础加速）

级	做什么	为什么
P0	增量 tokenizer 库（线性 BPE、紧凑表、SVE2 预分词）	最大一块 CPU 开销：每次 228 ms 核时 [68]
P0	KAE 卸载沙箱快照（4 KB 页，ZIP→SEC→SM3 链式）	客户点名；IAA 压快照 2–4.5× [89]
P0	KAE 做 zswap / zram 后端	不超卖时内存是密度第一上限
P0	DPA 用户态低时延接口，支持 SG 批量描述符	快照与写时复制都要批量搬移
P1	进程启动的用户态路径（RL 客户为 P0）	命令外开销均值 1.26 s [56]
P1	推理 host 分配器加大页后端	页表遍历占 LLC 停顿 85% [58]
P1	checkpoint、镜像、训练数据层接 KAE	加载数据占训练节点 40% CPU [53]
P2	SVE2 文本库、JSON 绑定、块哈希、网关 TLS	收益在 Python 路径或特定形态

960：硬件（按硅）

级	诉求	谁提
P0	核前端：L3→L1-I 取指带宽、BTB、ITLB	海思（本团队提供负载）
P0	KAE：4 KB 固定开销、预置 Huffman、链式、翻译不缺页	本团队
P0	DPA：小段 <100 ns、SG 描述符、与 KAE 共享地址空间	本团队
P0	TLBID（协作项）	海思、欧拉
P1	突发单核与 SKU 分档；SMT 与 MPAM；MOPS；跨寄存器搬运时延；内存容量与带宽取舍	海思（ISA 两项本团队提）
P2	L2 容量；核侧硬件队列；近核序列化引擎；其余 ISA 特性	海思

排序标准与两个说明

排序标准： 量级 × 鲲鹏差距 × 可行性 × 是否本团队的事；测量先于一切（第 63 页）。

量级最大的近核序列化引擎为什么排 P2： 软件零拷贝先拿走大头且已在阿里生产；格式与接口生态风险最高；网络与存储序列化的比例没有数据。

核侧硬件队列为什么排 P2： 文献收益上限高（nanoPU −5×、μManycore −10.4×），但只对 μs 级服务有效，软件生态缺失。

不做： tokenizer、JSON、正则的专用加速器；向量距离引擎；分配器硬件卸载；单独为 RPC 库做硬件。

这是全篇唯一的排序：第四部分只列文献的需求，取舍都在这里；“量级最大”不等于“先做”。

数据来源与口径：排序沿用整合文档表 4-1、4-3，并合并 v1 第 37 页（REPORT 6.4）的测量项；理由与数字见第 56–61 页。

在 950 上先测这 15 件事：每一件补矩阵里的一格——没有这些数据，前面的发力点都还是推断

#	测什么	回答什么问题	补矩阵哪一格	谁测
1	传统税的鲲鹏基线： DCPerf 、 HyperProtoBench 、 HyperCompressBench ；按调用栈把序列化、压缩、拷贝、分配拆到服务 / 存储 / 进程内三列	各行的量级；补对账矩阵的空格	传统各行 × 三列	基础加速、海思
2	tokenizer 基线： HuggingFace 、 tiktoken 、 fastokens 在 950 、 Graviton 4 、 EPYC 上的每核 MB/s 与 L2 MPKI	tokenize 税有多大、紧凑表省多少	tokenize × 模型列	基础加速
3	工具负载的前端分析：增量编译、 pytest 、 npm / pip 、 git 、 bash -c true 的 top-down	前端与突发单核的诉求有多强	前端 × 执行环境列	基础加速
4	进程创建： fork 随父进程大小、缺页时延、 microVM 内外的启动拆分	用户态能省多少、内核部分交给谁	冷启动 × 执行环境列	基础加速、欧拉
5	KAE 4 KB 页压缩：同步单页时延 P50 / P99 、 SVA 缺页、 1–64 KB 盈亏点	zswap 与快照压缩能否达到验收线	压缩 × 时间列	基础加速
6	快照保存与恢复：不压缩、软件压缩、 KAE 链式； 100 个沙箱并发检查点	暂停恢复卸载的收益	压缩、拷贝 × 执行环境列	基础加速、沙箱团队
7	KAE zswap ：沙箱匿名页的压缩比、换入时延、省下的 CPU	超卖能做到几倍	压缩 × 时间列	基础加速、欧拉
8	DPA 小段提交： 64 B–4 KB 的固定开销，与 SVE memcpy 的交叉点	DPA 能否承担 DSA 的角色	拷贝 × 服务 / 时间列	基础加速
9	MPAM 隔离：推理引擎加满负荷 tokenizer ，逐步加缓存与带宽分区	共置时怎么隔离	tokenize × 模型列（隔离）	基础加速
10	向量到标量的搬运： UMOV 、 FMOV 、 CNTP 等时延	跨寄存器时延诉求的依据	序列化 / tokenize 行	基础加速
11	推理 host 微架构： TLB 未命中、页表遍历占比、 graph dispatch 延迟	分配与调度税在鲲鹏上多大	分配、调度 × 模型列	推理框架、基础加速
12	沙箱密度与每瓦沙箱数：加沙箱直到高优先级 P99 超标； SMT 开关；每核带宽	内存与 CPU 哪个先成为上限	内存 × 时间	沙箱团队、基础加速
13	TLBI 扩展性： 1–48 台 VM 的创建、快照、迁页；核间 ping-pong	TLBID 诉求与 chiplet 的代价	执行环境列的系数	海思或欧拉
14	UB 基线：小消息 RTT 与 Mops/ 核； RC 对 RM （数千对端）； Mooncake 鲲鹏后端对 RDMA	UB 在 RPC 与 KV 路径上的立论	通信 × 服务列； KV 搬移	UB 团队、 KV 存储团队
15	命令级分布：自家编码 Agent 日志里 Bash 前 20 种命令的次数与 CPU 时间	git 、测试、构建各占多少	执行环境列的次数	基础加速

1 、 2 、 14 是基线：没有它们，传统税和 Agent 税在鲲鹏上的量级都只是推断； 4–8 决定本团队的 P0 能不能达到验收线。

数据来源与口径： REPORT.md 6.4 节待测清单；整合文档表 4-5 （ 19 项，此处合并为 15 项）。“补矩阵哪一格”对应第 21 页与第 47 页；部署形态占比与托管语言的序列化税要和客户联合测（第 24 页），不在本表。

附录 A：论文清单（[NN] 对应 notes/ 笔记编号；[L1–L4]、[R1–R10] 对应 ~/AgenticAI 阅读报告）

传统负载：用户提供（笔记 01–19）

Profiling a WSC (ISCA'15) · Cloud-Scale Characterization of RPCs (SOSP'23) · Lifting the veil on Meta's microservices (ATC'23) · Alibaba Trace Analysis (SoCC'21) · DCPerf (ISCA'25) · DeathStarBench (ASPLOS'19) · CRISP (ATC'22) · Homa (SIGCOMM'18) · R2P2 (ATC'19) · nanoPU (OSDI'21) · RPCValet (ASPLOS'19) · Dagger (ASPLOS'21) · Cerebros (MICRO'21) · HydraRPC (ATC'24) · LightRPC · Breakwater (OSDI'20) · Aequitas (SIGCOMM'22) · mRPC (NSDI'23) · zBuffer (PPoPP'26)

传统负载：补充（笔记 20–43）

Accelerometer · SoftSKU · Profiling Hyperscale Big Data (ISCA'23) · GreenSKU · Meta Hyperscale Infrastructure (CACM'25) · Memory Allocator at WSC (ASPLOS'24) · μManycore · Alibaba Complexity (arXiv'25) · ServiceRouter (OSDI'23) · MeshInsight · Canal Mesh (SIGCOMM'24) · XLB · Snap · Junction · Lightweight Sidecar for Serverless (SoCC'25) · eRPC · Falcon (SIGCOMM'25) · Homa/Linux · RpcNIC · RPCool · CEIO · FlexiNS · BenchBF3 · ProtoAcc · Optimus Prime · Cornflakes · Cereal · CDPU (ISCA'23) · SpecProto (ASPLOS'26) · Host Network Stack Overheads (SIGCOMM'21)

AI 与 Agent 负载（第五部分核心论文）

TraceLab (arXiv 2606.30560) [56] · AgentSysBench (arXiv 2608.15127) [57] · CPU-Centric Agentic AI (arXiv 2511.00739) [78] [L3] · Agentic AI Workload Characterization (IISWC'26) [L1] · Agentic Coding in the Wild: Copilot (arXiv 2608.00101) [R2] · CacheWise (arXiv 2606.16824) [R3] · AgentCgroup (arXiv 2602.09345) [84] [R4] · Not All AI Agents Are Equal (arXiv 2609.19947) [R5] · Aries (arXiv 2607.29069) [R6] · Energy on Blackwell (arXiv 2609.29707) [R7] · XPerf (arXiv 2608.20370) [R8] · DSec (arXiv 2609.22978) [R10] · TokTier (arXiv 2607.29678) [68] · SWE-MiniSandbox [76]

AI 与鲲鹏：整合文档引用的语料（笔记 44–112）

CloudMatrix384 · Mooncake · Splitwise · DistServe · NanoFlow · Firecracker · RAGO · UB-Mesh · xDeepServe · Blink · GH200 characterization · virtCCA · Parrot · CPU-underprovisioning (arXiv 2603.22774) · 闭门会 G1–G9 九组补充研究（密度、超卖、快照、嵌套虚拟化、TLB、轻量隔离、机密计算、UINTR / SMMU / DPU、安卓沙箱）

市场信号（第五部分开头）

Intel Q2 2026 新闻稿与 10-Q · AMD Q2 2026 新闻稿 · Arm Q1 FYE27 新闻稿 · NVIDIA Q2 FY27 财报（经媒体） · 海光 2026 半年报（经新华财经） · TrendForce 2026-04-14 《The Great Rebalance》 · Mercury Research Q2 2026 （经 HotHardware） · Nikkei Asia、Reuters 的交期报道 · OpenRouter （Crypto Briefing、The Decoder）

建议的下一步阅读

Swift (SIGCOMM'20) · PLB (SIGCOMM'22) · Ultra Ethernet 规范 · XFaaS (SOSP'23) · Karandikar 博士论文 (2024) · CXL 3.x 多主机共享的真实硬件评估 · 云端任务沙箱与企业托管运行时的生产负载数据（公开文献空白）

数据来源与口径：传统部分 PDF 在 RPC_PAPERS/ 与 downloaded/（前缀 A–F），全文在 text/，笔记在 notes/（总表 notes/INDEX.md）；第五部分新论文在 ~/AgenticAI 与 ~/AgenticAI/related/。图按 [NN] Fig. N 截自对应 PDF。

附录 B： 关键数字口径表——全篇反复出现的数字只用这一种说法

对象	标准表述	口径	主讲页
负载集中度	按 CPU 周期分散（top-1 约 10%、 top-50 约 60%）；按调用次数集中（top-100 方法 91%）；调用集中 ≠ 周期集中	周期对调用次数	16
税的矩阵	按操作切（行）是数据中心税，按边界切（列）是某条边界的税； RPC 税是服务那一列，两者只在交叉格重叠	本报告的组织方式	6
税的总量	Google 2015 六项 22–27%； Meta 核心应用逻辑最少 18%；存储与分析平台税 32–40%	fleet、 7 个服务、 3 个平台	9
内核	Google fleet 全部内核 18–20%； Meta 缓存类约 30% 时间；网络栈只是其中一部分	fleet 对单类服务	11
RPC 税	服务那一列： [02] 口径 7.1% （压缩 3.1、 TCP 1.7、序列化 1.2、库 1.1）；压缩只计 RPC 库发起的部分则约 4–5% （推算）	fleet 平均、两个口径	21
压缩	整行 2.9% [34]； RPC 库直接发起的占 13.9% （约 0.4%，推算）； [02] 记入 RPC 税的 3.1% 归类存疑	整行对一格	21
序列化与 RPC	protobuf 整行 9.6%，服务那一格约 1.2 个百分点（八分之一）；编解码操作里 16% / 35% 来自 RPC 栈	两种分母	23
每请求 RPC 数	Meta Fetch 中位 498 个服务块，中位深 4、宽 472， P99 宽约 7,400	生产 trace	15
形态系数	CPU： library 1.36×、 sidecar 2.7–3.7×；延迟： sidecar p50 +107%， serverless 加策略 p50 7×、 p99 254×	两条链分开	20
前端	front-end bound 占 slot 15–30%； L1-I MPKI 7–56；不与周期占比排序	slot 口径	11
SMT	吞吐有空间（IPC 不到峰值一半）；尾延迟代价 +45.2% （core scheduling 后 +17.3%）	两面	11
内存带宽	传统利用率中位 10%、 95 分位 31%； Agent 先撞容量，带宽未测	两类负载	59
编码 Agent 一次请求	8.8 次模型 + 10.8 次工具（均值）；每步中位 11.9 万 / 875 / 214 token；新内容约 0.8%	TraceLab	45
命中率与时间构成	命中率 95.7% （token 加权）；会话中位 20% 时间在执行；人的思考占 92.3%	三份数据	45
工具时间占比	2–89%： TraceLab 59.8% （本地机器、请求内）、 Copilot 4.7% （会话墙钟）	负载与口径	45
每个沙箱	平均约 0.1 核（0.08–0.13）；内存基线 185 MB + 2–4 GB 突发，峰均比 15.4×	单机与生产	44
冷启动	初始化 31–48% （编码任务）； RL 19–29%； Codex 命令外 1.26 s （中位 0.16 s）	受控与真实 trace	50
tokenize	每次 12 万 token： HF 228 ms、 Rust 13.4 ms、增量 2.3 ms 核时；“64%”只见于摘要	TokTier 表 8	49
Agent token 规模	OpenRouter 每天约 7.3 万亿（7 日均值， 2026-08-10）	第三方平台	43
CPU:GPU	各方说法： 1:4–1:8 → 接近 1:1 （Intel、 AMD）、 1:1–1:2 （TrendForce）；柜内 1:2 为内存；按生产密度与厂商规格差一个数量级	预测（利益相关方）	38
CPU 厂商业绩	Intel DCAI +59% （出货 +9%、售价 +48%）； AMD 数据中心 +107% （含 GPU）； Arm 数据中心版税翻倍以上；无一家单列 Agent	最近一期同比	39

数据来源与口径：完整的对象登记表（33 个对象，含来源与引用页）见 OUTLINE_RPC_DECK_V2.md 第 2 节； v1 的一致性问题清单见同一文件第 1 节。